Angular 10 (17 updated) Boilerplate - Email Sign Up with Verification, Authentication & Forgot Password

Tutorial built with Angular 17.3.2

In this tutorial we'll cover how to implement a boilerplate sign up and authentication system in Angular that includes:

- Email sign up and verification
- JWT authentication with refresh tokens
- Role based authorization with support for two roles (User & Admin)
- Forgot password and reset password functionality
- View and update my profile section
- Admin section with sub section for managing all accounts (restricted to the Admin role)

Angular Boilerplate App Overview

The Angular boilerplate app runs with a fake backend by default to enable it to run completely in the browser without a real backend api (backend-less), to switch to a real backend api you just have to remove a couple of lines of code from the app module file (/src/app/app.module.ts). You can build your own api or hook it up with one of the boilerplate apis available (Node.js + MySQL) with the instructions below.

There are no accounts registered in the application by default, in order to login you must first register and verify an account. The fake backend displays "email" messages on screen because it can't send real emails, so after registration a "verification email" is displayed with a link to verify the account just registered, click the link to verify the account and login to the Angular boilerplate app.

The first account registered is assigned to the Admin role and subsequent accounts are assigned to the regular User role. Admins can access the admin

section and manage all accounts, regular user accounts can only update their own profile.

JWT authentication with refresh tokens

Authentication is implemented with JWT access tokens and refresh tokens. On successful authentication the api (or fake backend) returns a short lived JWT access token that expires after 15 minutes, and a refresh token that expires after 7 days in a cookie. The JWT is used for accessing secure routes on the api and the refresh token is used for generating new JWT access tokens when (or just before) they expire, the Angular app starts a timer to refresh the JWT token 1 minute before it expires to keep the account logged in.

Angular 10 Boilerplate Project Structure

The Angular CLI was used to generate the base project structure with the `ng new <project name>` command, the CLI is also used to build and serve the boilerplate application. For more info about the Angular CLI see https://angular.io/cli.

The app and code structure of the tutorial mostly follow the best practice recommendations in the official Angular Style Guide, with a few of my own tweaks here and there.

Each feature has it's own folder (account, admin, home & profile), other shared/common code such as components, services, models, helpers etc are

placed in folders prefixed with an underscore `_` to easily differentiate them from features and group them together at the top of the folder structure.

The `index.ts` files in some folders are barrel files that group the exported modules from that folder together so they can be imported using only the folder path instead of the full module path, and to enable importing multiple modules in a single import (e.g. `import { AccountService, AlertService } from '@app/_services'`).

The `account`, `admin` and `profile` features are organized into self contained feature modules that manage their own layout, routes and components, and are hooked into the main app inside the app routing module with lazy loading.

Path aliases `@app` and `@environments` have been configured in tsconfig.base.json that map to the `/src/app` and `/src/environments` directories. This allows imports to be relative to the app and environments folders by prefixing import paths with aliases instead of having to use long relative paths (e.g. `import MyComponent from '../../../MyComponent'`).

Here are the main project files that contain the boilerplate application logic. I left out some files that were generated by Angular CLI `ng new` command that I didn't change.

- src
  - app
    - _components
      - alert.component.html
      - alert.component.ts
      - index.ts
    - _helpers
      - app.initializer.ts
      - auth.guard.ts
      - error.interceptor.ts
      - fake-backend.ts
      - jwt.interceptor.ts
      - must-match.validator.ts
      - index.ts
    - _models
      - account.ts
      - alert.ts
      - role.ts
      - index.ts
    - _services
      - account.service.ts
      - alert.service.ts
      - index.ts

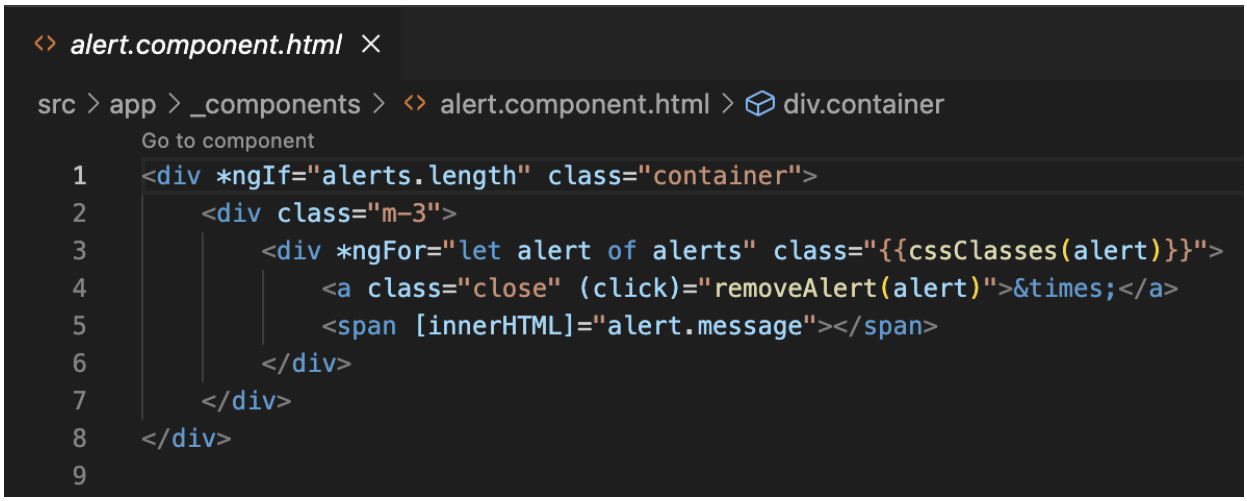
- account
  - account-routing.module.ts
  - account.module.ts
  - forgot-password.component.html
  - forgot-password.component.ts
  - layout.component.html
  - layout.component.ts
  - login.component.html
  - login.component.ts
  - register.component.html
  - register.component.ts
  - reset-password.component.html
  - reset-password.component.ts
  - verify-email.component.html
  - verify-email.component.ts
- admin
  - accounts
    - accounts-routing.module.ts
    - accounts.module.ts
    - add-edit.component.html
    - add-edit.component.ts
    - list.component.html
    - list.component.ts
  - admin-routing.module.ts
  - admin.module.ts
  - layout.component.html
  - layout.component.ts
  - overview.component.html
  - overview.component.ts
  - subnav.component.html

        - home
            - home.component.html
            - home.component.ts
            - index.ts
        - profile
            - details.component.html
            - details.component.ts
            - layout.component.html
            - layout.component.ts
            - profile-routing.module.ts
            - profile.module.ts
            - update.component.html
            - update.component.ts
        - app-routing.module.ts
        - app.component.html
        - app.component.ts
        - app.module.ts
    - environments
        - environment.prod.ts
        - environment.ts
    - index.html
    - main.ts
    - polyfills.ts
    - styles.less
- package.json
- tsconfig.base.json

Alert Component Template

Path: /src/app/_components/alert.component.html

The alert component template contains the html for displaying alert messages at the top of the page, it renders a notification for each alert in the `alerts` array of the alert component below.



Alert Component

Path: /src/app/_components/alert.component.ts

The alert component controls the adding & removing of alerts in the UI, it maintains an array of alerts that are rendered by the component template.

The component subscribes to receive new alerts from the alert service in the `ngOnInit` method by calling the `alertService.onAlert()` method; new alerts are added to the `alerts` array for display. Alerts are cleared when an alert with

an empty message is received from the alert service. The `ngOnInit` method also calls `router.events.subscribe()` to subscribe to route change events so it can automatically clear alerts on route changes.

The `ngOnDestroy()` method unsubscribes from the alert service and router events when the component is destroyed to prevent memory leaks from orphaned subscriptions.

The `removeAlert()` method removes the specified `alert` object from the array, which allows individual alerts to be closed in the UI.

The `cssClasses()` method returns a corresponding bootstrap alert class for each of the alert types, if you're using something other than bootstrap you can change the CSS classes returned to suit your application.

TS *alert.component.ts* ×

src > app > _components > TS alert.component.ts > ...

```ts
import { Component, OnInit, OnDestroy, Input } from '@angular/core';
import { Router, NavigationStart } from '@angular/router';
import { Subscription } from 'rxjs';

import { Alert, AlertType } from '@app/_models';
import { AlertService } from '@app/_services';

@Component({ selector: 'alert', templateUrl: 'alert.component.html' })
export class AlertComponent implements OnInit, OnDestroy {
    @Input() id = 'default-alert';
    @Input() fade = true;

    alerts: Alert[] = [];
    alertSubscription: Subscription;
    routeSubscription: Subscription;

    constructor(private router: Router, private alertService: AlertService) { }

```

```
    ngOnInit() {
        // subscribe to new alert notifications
        this.alertSubscription = this.alertService.onAlert(this.id)
            .subscribe(alert => {
                // clear alerts when an empty alert is received
                if (!alert.message) {
                    // filter out alerts without 'keepAfterRouteChange' flag
                    this.alerts = this.alerts.filter(x => x.keepAfterRouteChange);

                    // remove 'keepAfterRouteChange' flag on the rest
                    this.alerts.forEach(x => delete x.keepAfterRouteChange);
                    return;
                }

                // add alert to array
                this.alerts.push(alert);

                // auto close alert if required
                if (alert.autoClose) {
                    setTimeout(() => this.removeAlert(alert), 3000);
                }
            });

        // clear alerts on location change
        this.routeSubscription = this.router.events.subscribe(event => {
            if (event instanceof NavigationStart) {
                this.alertService.clear(this.id);
            }
        });
    }

```

```
    ngOnDestroy() {
        // unsubscribe to avoid memory leaks
        this.alertSubscription.unsubscribe();
        this.routeSubscription.unsubscribe();
    }

    removeAlert(alert: Alert) {
        // check if already removed to prevent error on auto close
        if (!this.alerts.includes(alert)) return;

        if (this.fade) {
            // fade out alert
            alert.fade = true;

            // remove alert after faded out
            setTimeout(() => {
                this.alerts = this.alerts.filter(x => x !== alert);
            }, 250);
        } else {
            // remove alert
            this.alerts = this.alerts.filter(x => x !== alert);
        }
    }

```

```
74      cssClasses(alert: Alert) {
75          if (!alert) return;
76
77          const classes = ['alert', 'alert-dismissable'];
78
79          const alertTypeClass = {
80              [AlertType.Success]: 'alert alert-success',
81              [AlertType.Error]: 'alert alert-danger',
82              [AlertType.Info]: 'alert alert-info',
83              [AlertType.Warning]: 'alert alert-warning'
84          }
85
86          classes.push(alertTypeClass[alert.type]);
87
88          if (alert.fade) {
89              classes.push('fade');
90          }
91
92          return classes.join(' ');
93      }
94  }
```

TS *index.ts* ×

src > app > _components > TS index.ts

```
1   export * from './alert.component';
```

App Initializer

Path: /src/app/_helpers/app.initializer.ts

The app initializer runs before the boilerplate app starts up and attempts to automatically authenticate the user in the background by calling `accountService.refreshToken()` to get a new JWT token from the api using the refresh token stored in the browser cookie (if it exists).

If the user has logged in previously (without logging out) and the browser still contains a valid refresh token cookie then they will be automatically logged in when the app loads.

The call to the `.subscribe()` method triggers the request to the api, and the `.add()` method is used for executing additional logic after the request completes (success or failure), so it works like a promise `finally()` method.

The app initializer is added to angular app in the `providers` section of the app module using the `APP_INITIALIZER` injection token. For more info see https://angular.io/api/core/APP_INITIALIZER.

```
1   import { AccountService } from '@app/_services';
2
3   export function appInitializer(accountService: AccountService) {
4       return () => new Promise(resolve => {
5           // attempt to refresh token on app start up to auto authenticate
6           accountService.refreshToken()
7               .subscribe()
8               .add(resolve);
9       });
10  }
```

Auth Guard

Path: /src/app/_helpers/auth.guard.ts

The auth guard is an angular route guard that's used to prevent unauthorized users from accessing restricted routes, it does this by implementing the `CanActivate` interface which allows the guard to decide if a route can be activated with the `canActivate()` method. If the method returns `true` the route is activated (allowed to proceed), otherwise if the method returns `false` the route is blocked.

The auth guard uses the account service to check if the user is logged in, if the user isn't logged in they're redirected to the `/login` page with the `returnUrl` in the query parameters, if the user is logged in but not in an authorized role they're redirected to the home page

Angular route guards are attached to routes in the router config, this auth guard is used in app-routing.module.ts to protect the home, profile and admin routes.

```
import { Injectable } from '@angular/core';
import { Router, ActivatedRouteSnapshot, RouterStateSnapshot } from '@angular/router';

import { AccountService } from '@app/_services';

@Injectable({ providedIn: 'root' })
export class AuthGuard  {
    constructor(
        private router: Router,
        private accountService: AccountService
    ) { }

    canActivate(route: ActivatedRouteSnapshot, state: RouterStateSnapshot) {
        const account = this.accountService.accountValue;
        if (account) {
            // check if route is restricted by role
            if (route.data.roles && !route.data.roles.includes(account.role)) {
                // role not authorized so redirect to home page
                this.router.navigate(['/']);
                return false;
            }

            // authorized so return true
            return true;
        }

        // not logged in so redirect to login page with the return url
        this.router.navigate(['/account/login'], { queryParams: { returnUrl: state.url }});
        return false;
    }
}
```

Error Interceptor

Path: /src/app/_helpers/error.interceptor.ts

The Error Interceptor intercepts http responses from the api to check if there were any errors. If there is a `401 Unauthorized` or `403 Forbidden` response the account is automatically logged out of the boilerplate application, all other errors are re-thrown up to the calling service so an alert with the error can be displayed on the screen.

It's implemented using the Angular `HttpInterceptor` interface included in the `HttpClientModule`, by implementing the HttpInterceptor interface you can create a custom interceptor to catch all error responses from the api in a single location.

Http interceptors are added to the request pipeline in the providers section of the app.module.ts file.

```
import { Injectable } from '@angular/core';
import { HttpRequest, HttpHandler, HttpEvent, HttpInterceptor } from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { catchError } from 'rxjs/operators';

import { AccountService } from '@app/_services';

@Injectable()
export class ErrorInterceptor implements HttpInterceptor {
    constructor(private accountService: AccountService) { }

    intercept(request: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
        return next.handle(request).pipe(catchError(err => {
            if ([401, 403].includes(err.status) && this.accountService.accountValue) {
                // auto logout if 401 or 403 response returned from api
                this.accountService.logout();
            }

            const error = (err && err.error && err.error.message) || err.statusText;
            console.error(err);
            return throwError(error);
        }))
    }
}
```

Fake Backend Provider

Path: /src/app/_helpers/fake-backend.ts

In order to run and test the Angular boilerplate app without a real backend API, the example uses a fake backend that intercepts the HTTP requests from

the Angular app and sends back "fake" responses. This is done by a class that implements the Angular `HttpInterceptor` interface, for more information on Angular HTTP Interceptors see https://angular.io/api/common/http/HttpInterceptor or this article.

The fake backend is organized into a top level `handleRoute()` function that checks the request url and method to determine how the request should be handled. For intercepted routes one of the below `// route functions` is called, for all other routes the request is passed through to the real backend by calling `next.handle(request)`. Below the route functions there are `// helper functions` for returning different response types and performing small tasks.

The fake backend can't send emails so instead displays "email" messages on the screen by calling `alertService.info()` with the contents of the email e.g. "verify email" and "reset password" emails.

A fake backend is used for doing backendless development in Angular which allows you to run and test your code without a real backend api, it's perfect for code hosted in StackBlitz which doesn't have a backend, or when you're developing a front end before the backend is available.

I created the below fake backend as part of an Angular 10 User Registration and Login Example & Tutorial, it's built by extending the Angular

`HttpInterceptor` interface and includes mock endpoints for authentication, registration and user management, and uses browser local storage to store data to mimic a real api with a database.

Passing through unmocked requests to a real backend API

Sometimes there's a need to pass through specific requests to the server instead of being caught by the fake backend, for example when the real backend is partially completed and has some endpoints available.

With Angular 10 HTTP interceptors this is done by calling `return next.handle(request);`, you can see the code for passing through unmocked requests in the `default` case of the `switch` statement below.

Angular 10 Fake Backend Provider

```
import { Injectable } from '@angular/core';
import { HttpRequest, HttpResponse, HttpHandler, HttpEvent, HttpInterceptor, HTTP_INTERCEPTORS, HttpHeaders } from '@angular/common/http';
import { Observable, of, throwError } from 'rxjs';
import { delay, materialize, dematerialize } from 'rxjs/operators';

import { AlertService } from '@app/_services';
import { Role } from '@app/_models';

// array in local storage for accounts
const accountsKey = 'angular-10-signup-verification-boilerplate-accounts';
let accounts = JSON.parse(localStorage.getItem(accountsKey)) || [];

@Injectable()
export class FakeBackendInterceptor implements HttpInterceptor {
    constructor(private alertService: AlertService) { }

    intercept(request: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
        const { url, method, headers, body } = request;
        const alertService = this.alertService;

        return handleRoute();

        function handleRoute() {
            switch (true) {
                case url.endsWith('/accounts/authenticate') && method === 'POST':
                    return authenticate();
                case url.endsWith('/accounts/refresh-token') && method === 'POST':
                    return refreshToken();
                case url.endsWith('/accounts/revoke-token') && method === 'POST':
                    return revokeToken();
                case url.endsWith('/accounts/register') && method === 'POST':
                    return register();
                case url.endsWith('/accounts/verify-email') && method === 'POST':
                    return verifyEmail();
                case url.endsWith('/accounts/forgot-password') && method === 'POST':
                    return forgotPassword();
                case url.endsWith('/accounts/validate-reset-token') && method === 'POST':
                    return validateResetToken();
                case url.endsWith('/accounts/reset-password') && method === 'POST':
                    return resetPassword();
                case url.endsWith('/accounts') && method === 'GET':
                    return getAccounts();
                case url.match(/\/accounts\/\d+$/) && method === 'GET':
                    return getAccountById();
                case url.endsWith('/accounts') && method === 'POST':
                    return createAccount();
                case url.match(/\/accounts\/\d+$/) && method === 'PUT':
                    return updateAccount();
                case url.match(/\/accounts\/\d+$/) && method === 'DELETE':
                    return deleteAccount();
                default:
                    // pass through any requests not handled above
                    return next.handle(request);
            }
        }

```

```
57        // route functions
58
59        function authenticate() {
60            const { email, password } = body;
61            const account = accounts.find(x => x.email === email && x.password === password && x.isVerified);
62
63            if (!account) return error('Email or password is incorrect');
64
65            // add refresh token to account
66            account.refreshTokens.push(generateRefreshToken());
67            localStorage.setItem(accountsKey, JSON.stringify(accounts));
68
69            return ok({
70                ...basicDetails(account),
71                jwtToken: generateJwtToken(account)
72            });
73        }
74
```

```
75            function refreshToken() {
76                const refreshToken = getRefreshToken();
77
78                if (!refreshToken) return unauthorized();
79
80                const account = accounts.find(x => x.refreshTokens.includes(refreshToken));
81
82                if (!account) return unauthorized();
83
84                // replace old refresh token with a new one and save
85                account.refreshTokens = account.refreshTokens.filter(x => x !== refreshToken);
86                account.refreshTokens.push(generateRefreshToken());
87                localStorage.setItem(accountsKey, JSON.stringify(accounts));
88
89                return ok({
90                    ...basicDetails(account),
91                    jwtToken: generateJwtToken(account)
92                });
93
94            }
95
96            function revokeToken() {
97                if (!isAuthenticated()) return unauthorized();
98
99                const refreshToken = getRefreshToken();
100               const account = accounts.find(x => x.refreshTokens.includes(refreshToken));
101
102               // revoke token and save
103               account.refreshTokens = account.refreshTokens.filter(x => x !== refreshToken);
104               localStorage.setItem(accountsKey, JSON.stringify(accounts));
105
106               return ok();
107           }
108
```

```
    function register() {
        const account = body;

        if (accounts.find(x => x.email === account.email)) {
            // display email already registered "email" in alert
            setTimeout(() => {
                alertService.info(`
                    <h4>Email Already Registered</h4>
                    <p>Your email ${account.email} is already registered.</p>
                    <p>If you don't know your password please visit the <a href="${location.origin}/account/forgot-password">forgot password</a> page.</p>
                    <div><strong>NOTE:</strong> The fake backend displayed this "email" so you can test without an api. A real backend would send a real email.</div>
                `, { autoClose: false });
            }, 1000);

            // always return ok() response to prevent email enumeration
            return ok();
        }

        // assign account id and a few other properties then save
        account.id = newAccountId();
        if (account.id === 1) {
            // first registered account is an admin
            account.role = Role.Admin;
        } else {
            account.role = Role.User;
        }
        account.dateCreated = new Date().toISOString();
        account.verificationToken = new Date().getTime().toString();
        account.isVerified = false;
        account.refreshTokens = [];
        delete account.confirmPassword;
        accounts.push(account);
        localStorage.setItem(accountsKey, JSON.stringify(accounts));

        // display verification email in alert
        setTimeout(() => {
            const verifyUrl = `${location.origin}/account/verify-email?token=${account.verificationToken}`;
            alertService.info(`
                <h4>Verification Email</h4>
                <p>Thanks for registering!</p>
                <p>Please click the below link to verify your email address:</p>
                <p><a href="${verifyUrl}">${verifyUrl}</a></p>
                <div><strong>NOTE:</strong> The fake backend displayed this "email" so you can test without an api. A real backend would send a real email.</div>
            `, { autoClose: false });
        }, 1000);

        return ok();
    }

    function verifyEmail() {
        const { token } = body;
        const account = accounts.find(x => !!x.verificationToken && x.verificationToken === token);

        if (!account) return error('Verification failed');

        // set is verified flag to true if token is valid
        account.isVerified = true;
        localStorage.setItem(accountsKey, JSON.stringify(accounts));

        return ok();
    }
```

```
        function forgotPassword() {
            const { email } = body;
            const account = accounts.find(x => x.email === email);

            // always return ok() response to prevent email enumeration
            if (!account) return ok();

            // create reset token that expires after 24 hours
            account.resetToken = new Date().getTime().toString();
            account.resetTokenExpires = new Date(Date.now() + 24*60*60*1000).toISOString();
            localStorage.setItem(accountsKey, JSON.stringify(accounts));

            // display password reset email in alert
            setTimeout(() => {
                const resetUrl = `${location.origin}/account/reset-password?token=${account.resetToken}`;
                alertService.info(`
                    <h4>Reset Password Email</h4>
                    <p>Please click the below link to reset your password, the link will be valid for 1 day:</p>
                    <p><a href="${resetUrl}">${resetUrl}</a></p>
                    <div><strong>NOTE:</strong> The fake backend displayed this "email" so you can test without an api. A real backend would send a real email.</div>
                `, { autoClose: false });
            }, 1000);

            return ok();
        }

        function validateResetToken() {
            const { token } = body;
            const account = accounts.find(x =>
                !!x.resetToken && x.resetToken === token &&
                new Date() < new Date(x.resetTokenExpires)
            );

            if (!account) return error('Invalid token');

            return ok();
        }
```

```
209         function resetPassword() {
210             const { token, password } = body;
211             const account = accounts.find(x =>
212                 !!x.resetToken && x.resetToken === token &&
213                 new Date() < new Date(x.resetTokenExpires)
214             );
215 
216             if (!account) return error('Invalid token');
217 
218             // update password and remove reset token
219             account.password = password;
220             account.isVerified = true;
221             delete account.resetToken;
222             delete account.resetTokenExpires;
223             localStorage.setItem(accountsKey, JSON.stringify(accounts));
224 
225             return ok();
226         }
227 
228         function getAccounts() {
229             if (!isAuthenticated()) return unauthorized();
230             return ok(accounts.map(x => basicDetails(x)));
231         }
232 
233         function getAccountById() {
234             if (!isAuthenticated()) return unauthorized();
235 
236             let account = accounts.find(x => x.id === idFromUrl());
237 
238             // user accounts can get own profile and admin accounts can get all profiles
239             if (account.id !== currentAccount().id && !isAuthorized(Role.Admin)) {
240                 return unauthorized();
241             }
242 
243             return ok(basicDetails(account));
244         }
```

```
246        function createAccount() {
247            if (!isAuthorized(Role.Admin)) return unauthorized();
248
249            const account = body;
250            if (accounts.find(x => x.email === account.email)) {
251                return error(`Email ${account.email} is already registered`);
252            }
253
254            // assign account id and a few other properties then save
255            account.id = newAccountId();
256            account.dateCreated = new Date().toISOString();
257            account.isVerified = true;
258            account.refreshTokens = [];
259            delete account.confirmPassword;
260            accounts.push(account);
261            localStorage.setItem(accountsKey, JSON.stringify(accounts));
262
263            return ok();
264        }
```

```
266        function updateAccount() {
267            if (!isAuthenticated()) return unauthorized();
268
269            let params = body;
270            let account = accounts.find(x => x.id === idFromUrl());
271
272            // user accounts can update own profile and admin accounts can update all profiles
273            if (account.id !== currentAccount().id && !isAuthorized(Role.Admin)) {
274                return unauthorized();
275            }
276
277            // only update password if included
278            if (!params.password) {
279                delete params.password;
280            }
281            // don't save confirm password
282            delete params.confirmPassword;
283
284            // update and save account
285            Object.assign(account, params);
286            localStorage.setItem(accountsKey, JSON.stringify(accounts));
287
288            return ok(basicDetails(account));
289        }
```

```
291         function deleteAccount() {
292             if (!isAuthenticated()) return unauthorized();
293 
294             let account = accounts.find(x => x.id === idFromUrl());
295 
296             // user accounts can delete own account and admin accounts can delete any account
297             if (account.id !== currentAccount().id && !isAuthorized(Role.Admin)) {
298                 return unauthorized();
299             }
300 
301             // delete account then save
302             accounts = accounts.filter(x => x.id !== idFromUrl());
303             localStorage.setItem(accountsKey, JSON.stringify(accounts));
304             return ok();
305         }
```

```
307         // helper functions
308 
309         function ok(body?) {
310             return of(new HttpResponse({ status: 200, body }))
311                 .pipe(delay(500)); // delay observable to simulate server api call
312         }
313 
314         function error(message) {
315             return throwError({ error: { message } })
316                 .pipe(materialize(), delay(500), dematerialize());
317                 // call materialize and dematerialize to ensure delay even if an error is thrown (https://github.com/Reactive-Extensions/RxJS/issues/648);
318         }
319 
320         function unauthorized() {
321             return throwError({ status: 401, error: { message: 'Unauthorized' } })
322                 .pipe(materialize(), delay(500), dematerialize());
323         }
324 
325         function basicDetails(account) {
326             const { id, title, firstName, lastName, email, role, dateCreated, isVerified } = account;
327             return { id, title, firstName, lastName, email, role, dateCreated, isVerified };
328         }
329 
330         function isAuthenticated() {
331             return !!currentAccount();
332         }
333 
334         function isAuthorized(role) {
335             const account = currentAccount();
336             if (!account) return false;
337             return account.role === role;
338         }
339 
340         function idFromUrl() {
341             const urlParts = url.split('/');
342             return parseInt(urlParts[urlParts.length - 1]);
343         }
```

```
345             function newAccountId() {
346                 return accounts.length ? Math.max(...accounts.map(x => x.id)) + 1 : 1;
347             }
348
349             function currentAccount() {
350                 // check if jwt token is in auth header
351                 const authHeader = headers.get('Authorization');
352                 if (!authHeader.startsWith('Bearer fake-jwt-token')) return;
353
354                 // check if token is expired
355                 const jwtToken = JSON.parse(atob(authHeader.split('.')[1]));
356                 const tokenExpired = Date.now() > (jwtToken.exp * 1000);
357                 if (tokenExpired) return;
358
359                 const account = accounts.find(x => x.id === jwtToken.id);
360                 return account;
361             }
362
363             function generateJwtToken(account) {
364                 // create token that expires in 15 minutes
365                 const tokenPayload = {
366                     exp: Math.round(new Date(Date.now() + 15*60*1000).getTime() / 1000),
367                     id: account.id
368                 }
369                 return `fake-jwt-token.${btoa(JSON.stringify(tokenPayload))}`;
370             }
371
372             function generateRefreshToken() {
373                 const token = new Date().getTime().toString();
374
375                 // add token cookie that expires in 7 days
376                 const expires = new Date(Date.now() + 7*24*60*60*1000).toUTCString();
377                 document.cookie = `fakeRefreshToken=${token}; expires=${expires}; path=/`;
378
379                 return token;
380             }
```

```
381
382             function getRefreshToken() {
383                 // get refresh token from cookie
384                 return (document.cookie.split(';').find(x => x.includes('fakeRefreshToken')) || '=').split('=')[1];
385             }
386         }
387     }
388
389     export let fakeBackendProvider = {
390         // use fake backend in place of Http service for backend-less development
391         provide: HTTP_INTERCEPTORS,
392         useClass: FakeBackendInterceptor,
393         multi: true
394     };
```

JWT Interceptor

Path: /src/app/_helpers/jwt.interceptor.ts

The JWT Interceptor intercepts http requests from the application to add a JWT auth token to the Authorization header if the user is logged in and the request is to the application api url (`environment.apiUrl`).

It's implemented using the `HttpInterceptor` interface included in the `HttpClientModule`, by implementing the HttpInterceptor interface you can create a custom interceptor to modify http requests before they get sent to the server.

Http interceptors are added to the request pipeline in the providers section of the app.module.ts file.

```
import { Injectable } from '@angular/core';
import { HttpRequest, HttpHandler, HttpEvent, HttpInterceptor } from '@angular/common/http';
import { Observable } from 'rxjs';

import { environment } from '@environments/environment';
import { AccountService } from '@app/_services';

@Injectable()
export class JwtInterceptor implements HttpInterceptor {
    constructor(private accountService: AccountService) { }

    intercept(request: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
        // add auth header with jwt if account is logged in and request is to the api url
        const account = this.accountService.accountValue;
        const isLoggedIn = account && account.jwtToken;
        const isApiUrl = request.url.startsWith(environment.apiUrl);
        if (isLoggedIn && isApiUrl) {
            request = request.clone({
                setHeaders: { Authorization: `Bearer ${account.jwtToken}` }
            });
        }

        return next.handle(request);
    }
}
```

Must Match Validator

Path: /src/app/_helpers/must-match.validator.ts

The reactive forms custom `MustMatch` validator is used by some components in the boilerplate app to validate that password and confirmPassword fields match, e.g. in the register component).

TS must-match.validator.ts × TS index.ts

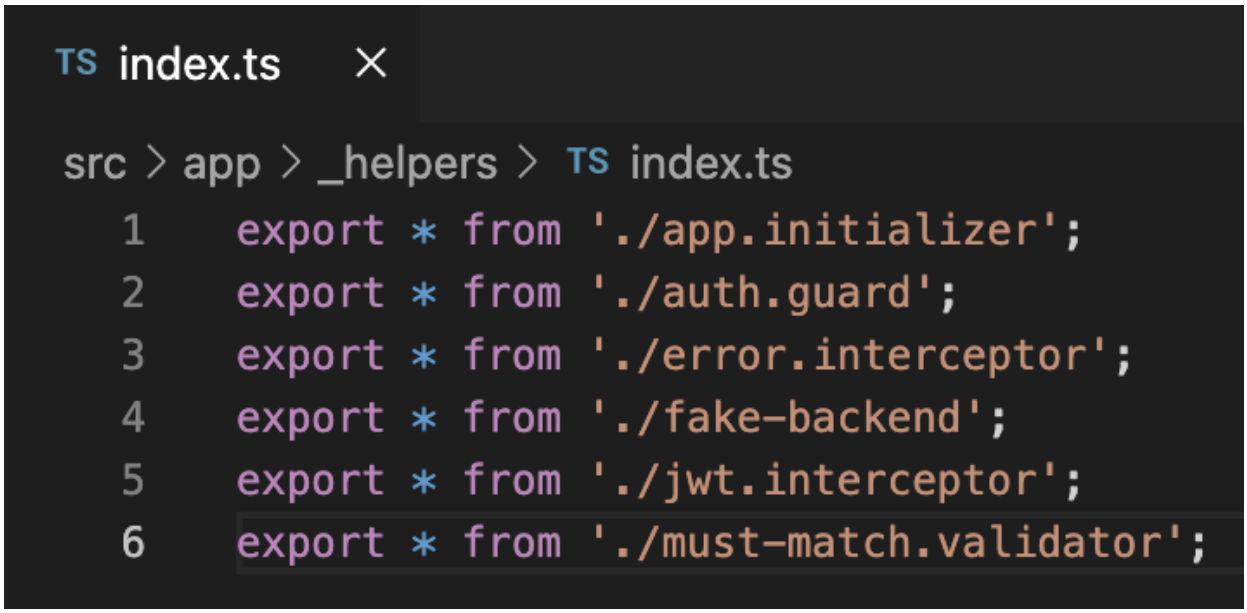
src > app > _helpers > TS must-match.validator.ts > MustMatch

```ts
import { UntypedFormGroup } from '@angular/forms';

// custom validator to check that two fields match
export function MustMatch(controlName: string, matchingControlName: string) {
    return (formGroup: UntypedFormGroup) => {
        const control = formGroup.controls[controlName];
        const matchingControl = formGroup.controls[matchingControlName];

        if (matchingControl.errors && !matchingControl.errors.mustMatch) {
            // return if another validator has already found an error on the matchingControl
            return;
        }

        // set error on matchingControl if validation fails
        if (control.value !== matchingControl.value) {
            matchingControl.setErrors({ mustMatch: true });
        } else {
            matchingControl.setErrors(null);
        }
    }
}
```



Account Model

Path: /src/app/_models/account.ts

The account model is a small class that defines the properties of an account.

```
1   import { Role } from './role';
2
3   export class Account {
4       id: string;
5       title: string;
6       firstName: string;
7       lastName: string;
8       email: string;
9       role: Role;
10      jwtToken?: string;
11  }
```

Alert Model and Alert Type Enum

Path: /src/app/_models/alert.ts

The `Alert` model defines the properties of each alert object, and the `AlertType` enum defines the types of alerts allowed in the application.

```ts
export class Alert {
    id: string;
    type: AlertType;
    message: string;
    autoClose: boolean;
    keepAfterRouteChange: boolean;
    fade: boolean;

    constructor(init?:Partial<Alert>) {
        Object.assign(this, init);
    }
}

export enum AlertType {
    Success,
    Error,
    Info,
    Warning
}
```

Role Enum

Path: /src/app/_models/role.ts

The role enum defines the roles that are supported by the application.

```ts
export enum Role {
    User = 'User',
    Admin = 'Admin'
}
```

TS index.ts

src > app > _models > TS index.ts

```ts
export * from './account';
export * from './alert';
export * from './role';
```

Account Service

Path: /src/app/_services/account.service.ts

The account service handles communication between the Angular boilerplate app and the backend api for everything related to accounts. It contains methods for the sign up, verification, authentication, refresh token and forgotten password flows, as well as standard CRUD methods for retrieving and modifying account data.

On successful login the api returns the account details and a JWT token which are published to all subscribers with the call to `this.accountSubject.next(account)`, the api also returns a refresh token cookie which is stored by the browser. The method then starts a countdown timer by calling `this.startRefreshTokenTimer()` to auto refresh the JWT token in the background (silent refresh) one minute before it expires in order to keep the account logged in.

The `logout()` method makes a POST request to the API to revoke the refresh token that is stored in the `refreshToken` cookie in the browser, cancels the silent refresh running in the background by calling `this.stopRefreshTokenTimer()`, then logs the user out by publishing a null value to all subscriber components (`this.accountSubject.next(null)`), and finally redirects to the login page.

The `account` property exposes an RxJS observable (`Observable<Account>`) so any component can subscribe to be notified when a user logs in, logs out, has their token refreshed or updates their profile. The notification is triggered by the call to `this.accountSubject.next()` from each of the corresponding methods in the service.

TS account.service.ts

src > app > _services > TS account.service.ts > ...

```ts
import { Injectable } from '@angular/core';
import { Router } from '@angular/router';
import { HttpClient } from '@angular/common/http';
import { BehaviorSubject, Observable } from 'rxjs';
import { map, finalize } from 'rxjs/operators';

import { environment } from '@environments/environment';
import { Account } from '@app/_models';

const baseUrl = `${environment.apiUrl}/accounts`;

@Injectable({ providedIn: 'root' })
export class AccountService {
    private accountSubject: BehaviorSubject<Account>;
    public account: Observable<Account>;

    constructor(
        private router: Router,
        private http: HttpClient
    ) {
        this.accountSubject = new BehaviorSubject<Account>(null);
        this.account = this.accountSubject.asObservable();
    }

    public get accountValue(): Account {
        return this.accountSubject.value;
    }

    login(email: string, password: string) {
        return this.http.post<any>(`${baseUrl}/authenticate`, { email, password }, { withCredentials: true })
            .pipe(map(account => {
                this.accountSubject.next(account);
                this.startRefreshTokenTimer();
                return account;
            }));
    }

```

```
    logout() {
        this.http.post<any>(`${baseUrl}/revoke-token`, {}, { withCredentials: true }).subscribe();
        this.stopRefreshTokenTimer();
        this.accountSubject.next(null);
        this.router.navigate(['/account/login']);
    }

    refreshToken() {
        return this.http.post<any>(`${baseUrl}/refresh-token`, {}, { withCredentials: true })
            .pipe(map((account) => {
                this.accountSubject.next(account);
                this.startRefreshTokenTimer();
                return account;
            }));
    }

    register(account: Account) {
        return this.http.post(`${baseUrl}/register`, account);
    }

    verifyEmail(token: string) {
        return this.http.post(`${baseUrl}/verify-email`, { token });
    }

    forgotPassword(email: string) {
        return this.http.post(`${baseUrl}/forgot-password`, { email });
    }

    validateResetToken(token: string) {
        return this.http.post(`${baseUrl}/validate-reset-token`, { token });
    }

    resetPassword(token: string, password: string, confirmPassword: string) {
        return this.http.post(`${baseUrl}/reset-password`, { token, password, confirmPassword });
    }

```

```
74      getAll() {
75          return this.http.get<Account[]>(baseUrl);
76      }
77
78      getById(id: string) {
79          return this.http.get<Account>(`${baseUrl}/${id}`);
80      }
81
82      create(params) {
83          return this.http.post(baseUrl, params);
84      }
85
86      update(id, params) {
87          return this.http.put(`${baseUrl}/${id}`, params)
88              .pipe(map((account: any) => {
89                  // update the current account if it was updated
90                  if (account.id === this.accountValue.id) {
91                      // publish updated account to subscribers
92                      account = { ...this.accountValue, ...account };
93                      this.accountSubject.next(account);
94                  }
95                  return account;
96              }));
97      }
98
99      delete(id: string) {
100         return this.http.delete(`${baseUrl}/${id}`)
101             .pipe(finalize(() => {
102                 // auto logout if the logged in account was deleted
103                 if (id === this.accountValue.id)
104                     this.logout();
105             }));
106     }
107
```

```
108      // helper methods
109
110      private refreshTokenTimeout;
111
112      private startRefreshTokenTimer() {
113          // parse json object from base64 encoded jwt token
114          const jwtToken = JSON.parse(atob(this.accountValue.jwtToken.split('.')[1]));
115
116          // set a timeout to refresh the token a minute before it expires
117          const expires = new Date(jwtToken.exp * 1000);
118          const timeout = expires.getTime() - Date.now() - (60 * 1000);
119          this.refreshTokenTimeout = setTimeout(() => this.refreshToken().subscribe(), timeout);
120      }
121
122      private stopRefreshTokenTimer() {
123          clearTimeout(this.refreshTokenTimeout);
124      }
125  }
```

Alert Service

Path: /src/app/_services/alert.service.ts

The alert service acts as the bridge between any component in an Angular boilerplate application and the alert component that actually displays the alert messages. It contains methods for sending, clearing and subscribing to alert messages.

You can trigger alert notifications from any component or service by calling one of the convenience methods for displaying the different types of alerts: `success()`, `error()`, `info()` and `warn()`.

Alert convenience method parameters

- The first parameter is the alert `message` string, which can be plain text or HTML

- The second parameter is an optional `options` object that supports an `autoClose` boolean property and `keepAfterRouteChange` boolean property:
    - `autoClose` - if true tells the alert component to automatically close the alert after three seconds. Default is `true`.
    - `keepAfterRouteChange` - if true prevents the alert from being closed after one route change, this is handy for displaying messages after a redirect such as when a new account is created or updated. Default is `false`.

The alert service uses the RxJS Observable and Subject classes to enable communication with other components,

TS alert.service.ts

src > app > _services > TS alert.service.ts > ...

```ts
import { Injectable } from '@angular/core';
import { Observable, Subject } from 'rxjs';
import { filter } from 'rxjs/operators';

import { Alert, AlertType } from '@app/_models';

@Injectable({ providedIn: 'root' })
export class AlertService {
    private subject = new Subject<Alert>();
    private defaultId = 'default-alert';

    // enable subscribing to alerts observable
    onAlert(id = this.defaultId): Observable<Alert> {
        return this.subject.asObservable().pipe(filter(x => x && x.id === id));
    }

    // convenience methods
    success(message: string, options?: any) {
        this.alert(new Alert({ ...options, type: AlertType.Success, message }));
    }

    error(message: string, options?: any) {
        this.alert(new Alert({ ...options, type: AlertType.Error, message }));
    }

    info(message: string, options?: any) {
        this.alert(new Alert({ ...options, type: AlertType.Info, message }));
    }

    warn(message: string, options?: any) {
        this.alert(new Alert({ ...options, type: AlertType.Warning, message }));
    }

    // core alert method
    alert(alert: Alert) {
        alert.id = alert.id || this.defaultId;
        alert.autoClose = (alert.autoClose === undefined ? true : alert.autoClose);
        this.subject.next(alert);
    }
```

```
40
41      // clear alerts
42      clear(id = this.defaultId) {
43          this.subject.next(new Alert({ id }));
44      }
45  }
```

TS index.ts

src > app > _services > TS index.ts

```
1  export * from './account.service';
2  export * from './alert.service';
```

Account Routing Module

Path: /src/app/account/account-routing.module.ts

The account routing module defines the routes for the account feature module. It includes routes for login, registration and related functionality, and a parent route for the account layout component which contains the common layout code for the account section.

TS account-routing.module.ts

src > app > account > TS account-routing.module.ts > ...

```ts
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { LayoutComponent } from './layout.component';
import { LoginComponent } from './login.component';
import { RegisterComponent } from './register.component';
import { VerifyEmailComponent } from './verify-email.component';
import { ForgotPasswordComponent } from './forgot-password.component';
import { ResetPasswordComponent } from './reset-password.component';

const routes: Routes = [
    {
        path: '', component: LayoutComponent,
        children: [
            { path: 'login', component: LoginComponent },
            { path: 'register', component: RegisterComponent },
            { path: 'verify-email', component: VerifyEmailComponent },
            { path: 'forgot-password', component: ForgotPasswordComponent },
            { path: 'reset-password', component: ResetPasswordComponent }
        ]
    }
];

@NgModule({
    imports: [RouterModule.forChild(routes)],
    exports: [RouterModule]
})
export class AccountRoutingModule { }
```

Account Module

Path: /src/app/account/account.module.ts

The account module defines the feature module for the account section along with metadata about the module. The `imports` specify which other angular modules are required by this module, and the `declarations` specify

which components belong to this module. For more info about angular 10 modules see https://angular.io/docs/ts/latest/guide/ngmodule.html.

The account module is hooked into the main app inside the app routing module with lazy loading.

TS account.module.ts ×

src > app > account > TS account.module.ts > ...

```ts
import { NgModule } from '@angular/core';
import { ReactiveFormsModule } from '@angular/forms';
import { CommonModule } from '@angular/common';

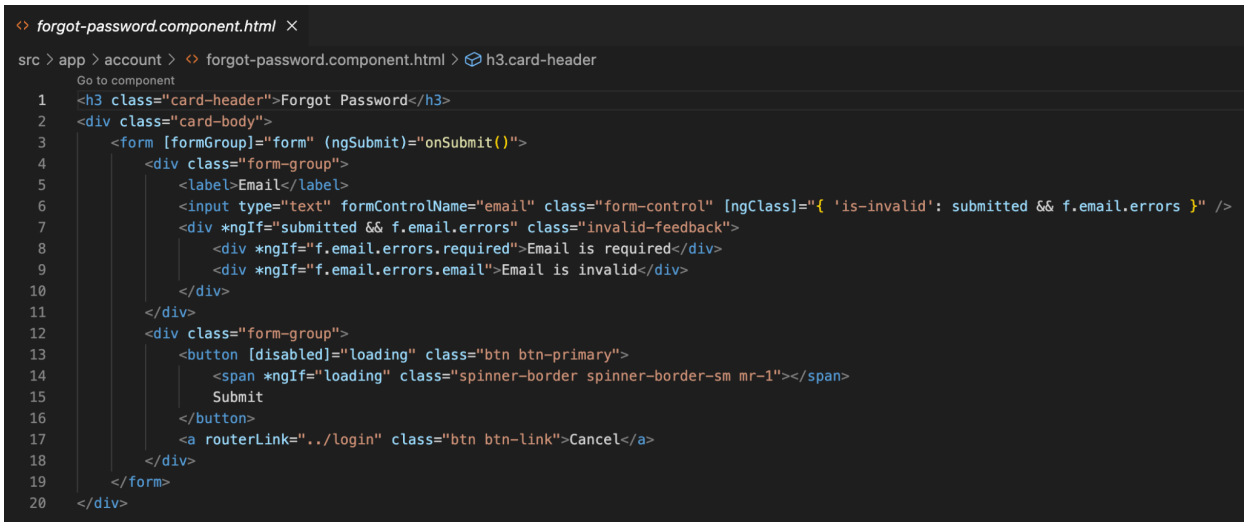
import { AccountRoutingModule } from './account-routing.module';
import { LayoutComponent } from './layout.component';
import { LoginComponent } from './login.component';
import { RegisterComponent } from './register.component';
import { VerifyEmailComponent } from './verify-email.component';
import { ForgotPasswordComponent } from './forgot-password.component';
import { ResetPasswordComponent } from './reset-password.component';

@NgModule({
    imports: [
        CommonModule,
        ReactiveFormsModule,
        AccountRoutingModule
    ],
    declarations: [
        LayoutComponent,
        LoginComponent,
        RegisterComponent,
        VerifyEmailComponent,
        ForgotPasswordComponent,
        ResetPasswordComponent
    ]
})
export class AccountModule { }
```

Forgot Password Component Template

Path: /src/app/account/forgot-password.component.html

The forgot password component template contains a simple form with a single field for entering the email of the account that you have forgotten password for. The form element uses the `[formGroup]` directive to bind to the `form` FormGroup in the forgot password component below, and it binds the form submit event to the `onSubmit()` handler in the forgot password component using the angular event binding `(ngSubmit)="onSubmit()"`.



Forgot Password Component

Path: /src/app/account/forgot-password.component.ts

On valid submit the forgot password component calls `this.accountService.forgotPassword()` and displays either a success or

error message. If the email matches a registered account the fake backend displays a password reset "email" with instructions in the UI below the success message (A real backend api would send an actual email for this step), the instructions include a link to reset the password of the account.

TS *forgot-password.component.ts* ×

src > app > account > TS forgot-password.component.ts > ...

```ts
import { Component, OnInit } from '@angular/core';
import { UntypedFormBuilder, UntypedFormGroup, Validators } from '@angular/forms';
import { first, finalize } from 'rxjs/operators';

import { AccountService, AlertService } from '@app/_services';

@Component({ templateUrl: 'forgot-password.component.html' })
export class ForgotPasswordComponent implements OnInit {
    form: UntypedFormGroup;
    loading = false;
    submitted = false;

    constructor(
        private formBuilder: UntypedFormBuilder,
        private accountService: AccountService,
        private alertService: AlertService
    ) { }

    ngOnInit() {
        this.form = this.formBuilder.group({
            email: ['', [Validators.required, Validators.email]]
        });
    }

    // convenience getter for easy access to form fields
    get f() { return this.form.controls; }

    onSubmit() {
        this.submitted = true;

        // reset alerts on submit
        this.alertService.clear();

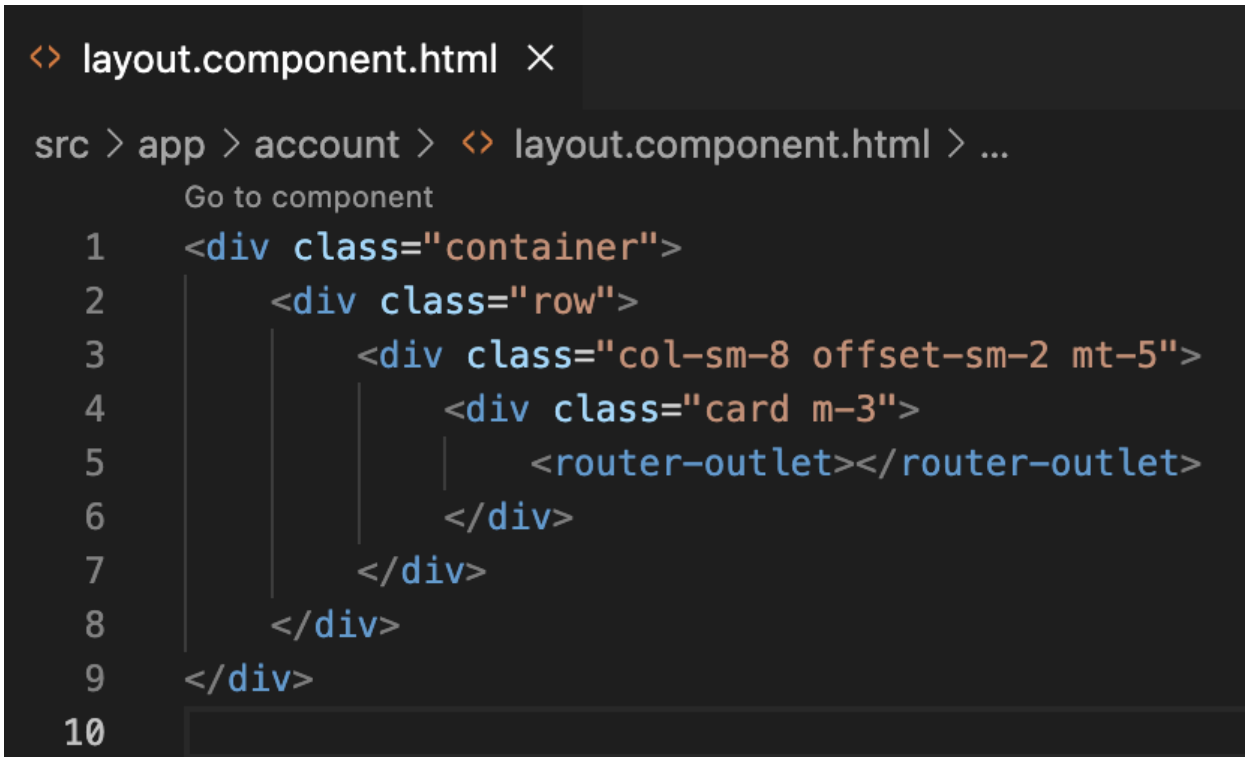
        // stop here if form is invalid
        if (this.form.invalid) {
            return;
        }

        this.loading = true;
        this.alertService.clear();
        this.accountService.forgotPassword(this.f.email.value)
            .pipe(first())
            .pipe(finalize(() => this.loading = false))
            .subscribe({
                next: () => this.alertService.success('Please check your email for password reset instructions'),
                error: error => this.alertService.error(error)
            });
    }
}
```

Account Layout Component Template

Path: /src/app/account/layout.component.html

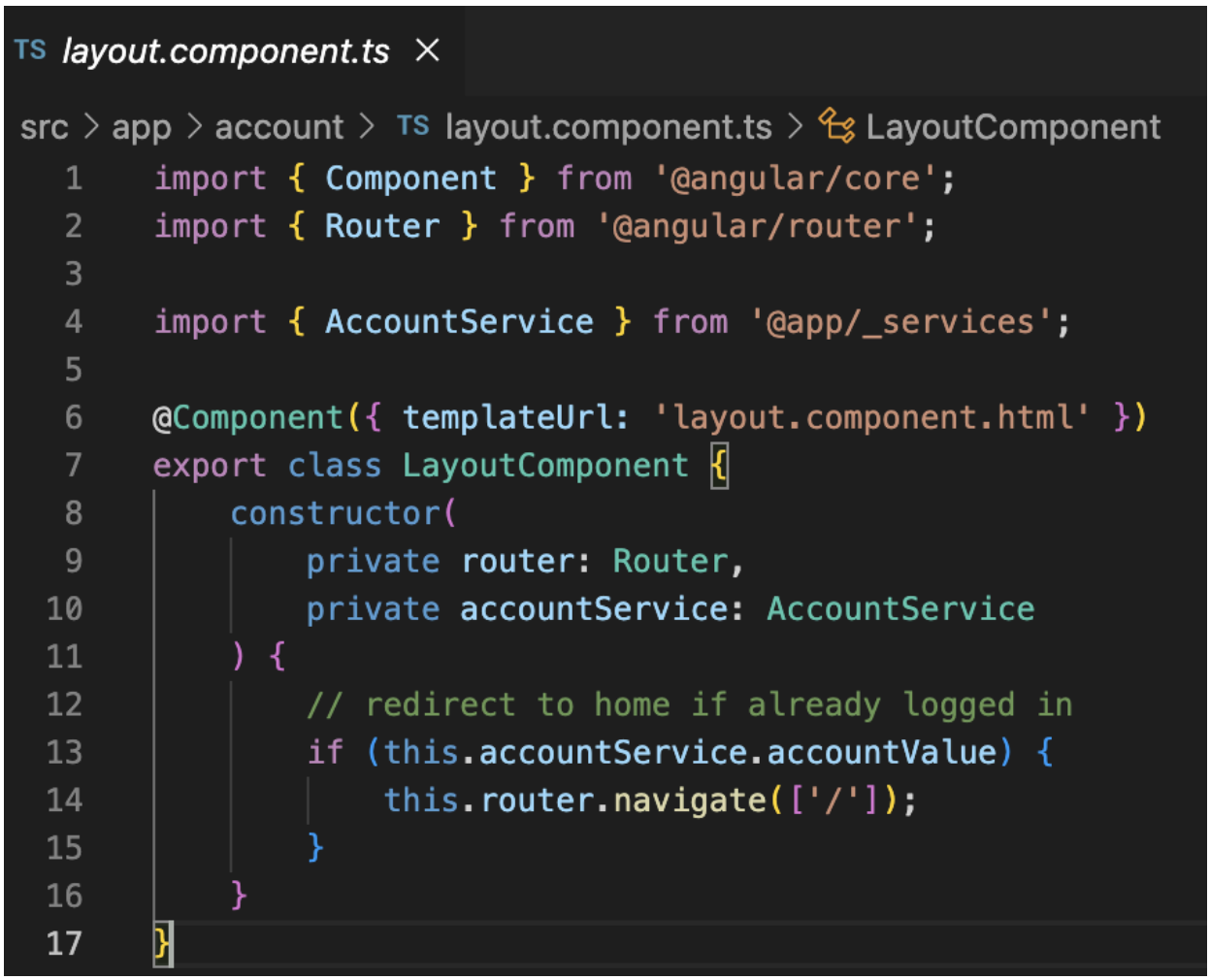
The account layout component template is the root template of the account feature / section of the boilerplate app, it contains the outer HTML for account registration, authentication and verification pages, and a `<router-outlet>` for rendering the currently routed component.



Account Layout Component

Path: /src/app/account/layout.component.ts

The account layout component is the root component of the account feature / section of the boilerplate app, it is bound to the account layout template with the `templateUrl` property of the angular `@Component` decorator, and automatically redirects the user to the home page if they are already logged in.

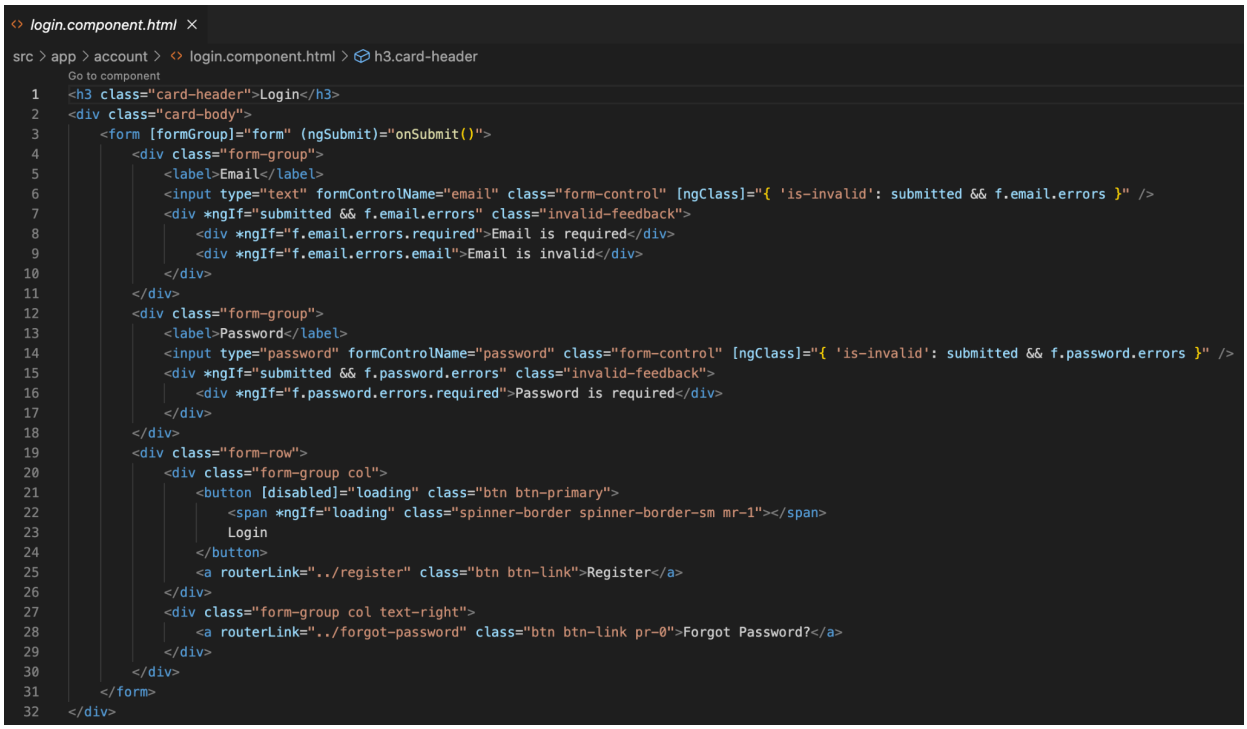


Login Component Template

Path: /src/app/account/login.component.html

The login component template contains a login form with email and password fields. It displays validation messages for invalid fields when the form is submitted. The form submit event is bound to the `onSubmit()` method of the login component.

The component uses reactive form validation to validate the input fields,



Login Component

Path: /src/app/account/login.component.ts

The login component uses the account service to login to the application on form submit. It creates the form fields and validators using an Angular `FormBuilder` to create an instance of a `FormGroup` that is stored in the `form`

property. The `form` is then bound to the `<form>` element in the login component template above using the `[formGroup]` directive.

On successful login the user is redirected to the page they were trying to access before logging in or to the home page (`'/'`) by default. The `returnUrl` query parameter is added to the url when redirected by the auth guard. On failed login the error returned from the backend is displayed in the UI.

The component contains a convenience getter property `f` to make it a bit easier to access form controls, for example you can access the password field in the template using `f.password` instead of `form.controls.password`.

TS login.component.ts

src > app > account > TS login.component.ts > ...

```ts
import { Component, OnInit } from '@angular/core';
import { Router, ActivatedRoute } from '@angular/router';
import { UntypedFormBuilder, UntypedFormGroup, Validators } from '@angular/forms';
import { first } from 'rxjs/operators';

import { AccountService, AlertService } from '@app/_services';

@Component({ templateUrl: 'login.component.html' })
export class LoginComponent implements OnInit {
    form: UntypedFormGroup;
    loading = false;
    submitted = false;

    constructor(
        private formBuilder: UntypedFormBuilder,
        private route: ActivatedRoute,
        private router: Router,
        private accountService: AccountService,
        private alertService: AlertService
    ) { }

    ngOnInit() {
        this.form = this.formBuilder.group({
            email: ['', [Validators.required, Validators.email]],
            password: ['', Validators.required]
        });
    }

    // convenience getter for easy access to form fields
    get f() { return this.form.controls; }

```

```
    onSubmit() {
        this.submitted = true;

        // reset alerts on submit
        this.alertService.clear();

        // stop here if form is invalid
        if (this.form.invalid) {
            return;
        }

        this.loading = true;
        this.accountService.login(this.f.email.value, this.f.password.value)
            .pipe(first())
            .subscribe({
                next: () => {
                    // get return url from query parameters or default to home page
                    const returnUrl = this.route.snapshot.queryParams['returnUrl'] || '/';
                    this.router.navigateByUrl(returnUrl);
                },
                error: error => {
                    this.alertService.error(error);
                    this.loading = false;
                }
            });
    }
}
```

Register Component Template

Path: /src/app/account/register.component.html

The register component template contains an account registration form with fields for title, first name, last name, email, password, confirm password and an accept Ts & Cs checkbox. All fields are required including the checkbox, the email field must be a valid email address, the password field must have a min length of 6 and must match the confirm password field.

The template displays validation messages for invalid fields when the form is submitted. The form element uses the `[formGroup]` directive to bind to the

`form` FormGroup in the register component below, and it binds the form submit event to the `onSubmit()` handler in the register component using the angular event binding `(ngSubmit)="onSubmit()"`.

The component uses reactive form validation to validate the input fields,

register.component.html

src > app > account > register.component.html > h3.card-header

Go to component

```
<h3 class="card-header">Register</h3>
<div class="card-body">
    <form [formGroup]="form" (ngSubmit)="onSubmit()">
        <div class="form-row">
            <div class="form-group col">
                <label>Title</label>
                <select formControlName="title" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.title.errors }">
                    <option value=""></option>
                    <option value="Mr">Mr</option>
                    <option value="Mrs">Mrs</option>
                    <option value="Miss">Miss</option>
                    <option value="Ms">Ms</option>
                </select>
                <div *ngIf="submitted && f.title.errors" class="invalid-feedback">
                    <div *ngIf="f.title.errors.required">Title is required</div>
                </div>
            </div>
            <div class="form-group col-5">
                <label>First Name</label>
                <input type="text" formControlName="firstName" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.firstName.errors }" />
                <div *ngIf="submitted && f.firstName.errors" class="invalid-feedback">
                    <div *ngIf="f.firstName.errors.required">First Name is required</div>
                </div>
            </div>
            <div class="form-group col-5">
                <label>Last Name</label>
                <input type="text" formControlName="lastName" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.lastName.errors }" />
                <div *ngIf="submitted && f.lastName.errors" class="invalid-feedback">
                    <div *ngIf="f.lastName.errors.required">Last Name is required</div>
                </div>
            </div>
        </div>
        <div class="form-group">
            <label>Email</label>
            <input type="text" formControlName="email" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.email.errors }" />
            <div *ngIf="submitted && f.email.errors" class="invalid-feedback">
                <div *ngIf="f.email.errors.required">Email is required</div>
                <div *ngIf="f.email.errors.email">Email must be a valid email address</div>
            </div>
        </div>
```

```
        <div class="form-row">
            <div class="form-group col">
                <label>Password</label>
                <input type="password" formControlName="password" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.password.errors }" />
                <div *ngIf="submitted && f.password.errors" class="invalid-feedback">
                    <div *ngIf="f.password.errors.required">Password is required</div>
                    <div *ngIf="f.password.errors.minlength">Password must be at least 6 characters</div>
                </div>
            </div>
            <div class="form-group col">
                <label>Confirm Password</label>
                <input type="password" formControlName="confirmPassword" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.confirmPassword.errors }" />
                <div *ngIf="submitted && f.confirmPassword.errors" class="invalid-feedback">
                    <div *ngIf="f.confirmPassword.errors.required">Confirm Password is required</div>
                    <div *ngIf="f.confirmPassword.errors.mustMatch">Passwords must match</div>
                </div>
            </div>
        </div>
        <div class="form-group form-check">
            <input type="checkbox" formControlName="acceptTerms" id="acceptTerms" class="form-check-input" [ngClass]="{ 'is-invalid': submitted && f.acceptTerms.errors }" />
            <label for="acceptTerms" class="form-check-label">Accept Terms & Conditions</label>
            <div *ngIf="submitted && f.acceptTerms.errors" class="invalid-feedback">Accept Ts & Cs is required</div>
        </div>
        <div class="form-group">
            <button [disabled]="loading" class="btn btn-primary">
                <span *ngIf="loading" class="spinner-border spinner-border-sm mr-1"></span>
                Register
            </button>
            <a routerLink="../login" href="" class="btn btn-link">Cancel</a>
        </div>
    </form>
</div>
```

Register Component

Path: /src/app/account/register.component.ts

The register component creates a new account with the account service when the register form is valid and submitted.

It creates the form fields and validators using an Angular `FormBuilder` to create an instance of a `FormGroup` that is stored in the `form` property. The `form` is then bound to the `<form>` element in the register component template using the `[formGroup]` directive.

The component contains a convenience getter property `f` to make it a bit easier to access form controls, for example you can access the password field in the template using `f.password` instead of `form.controls.password`.

On successful registration a success message is displayed and the user is redirected to the login page, then the fake backend displays a verification "email" with instructions in the UI below the success message (A real backend api would send an actual email for this step), the instructions include a link to verify the account.

TS *register.component.ts* ×

src > app > account > TS register.component.ts > ...

```ts
import { Component, OnInit } from '@angular/core';
import { Router, ActivatedRoute } from '@angular/router';
import { UntypedFormBuilder, UntypedFormGroup, Validators } from '@angular/forms';
import { first } from 'rxjs/operators';

import { AccountService, AlertService } from '@app/_services';
import { MustMatch } from '@app/_helpers';

@Component({ templateUrl: 'register.component.html' })
export class RegisterComponent implements OnInit {
    form: UntypedFormGroup;
    loading = false;
    submitted = false;

    constructor(
        private formBuilder: UntypedFormBuilder,
        private route: ActivatedRoute,
        private router: Router,
        private accountService: AccountService,
        private alertService: AlertService
    ) { }

    ngOnInit() {
        this.form = this.formBuilder.group({
            title: ['', Validators.required],
            firstName: ['', Validators.required],
            lastName: ['', Validators.required],
            email: ['', [Validators.required, Validators.email]],
            password: ['', [Validators.required, Validators.minLength(6)]],
            confirmPassword: ['', Validators.required],
            acceptTerms: [false, Validators.requiredTrue]
        }, {
            validator: MustMatch('password', 'confirmPassword')
        });
    }

    // convenience getter for easy access to form fields
    get f() { return this.form.controls; }

```

```
    onSubmit() {
        this.submitted = true;

        // reset alerts on submit
        this.alertService.clear();

        // stop here if form is invalid
        if (this.form.invalid) {
            return;
        }

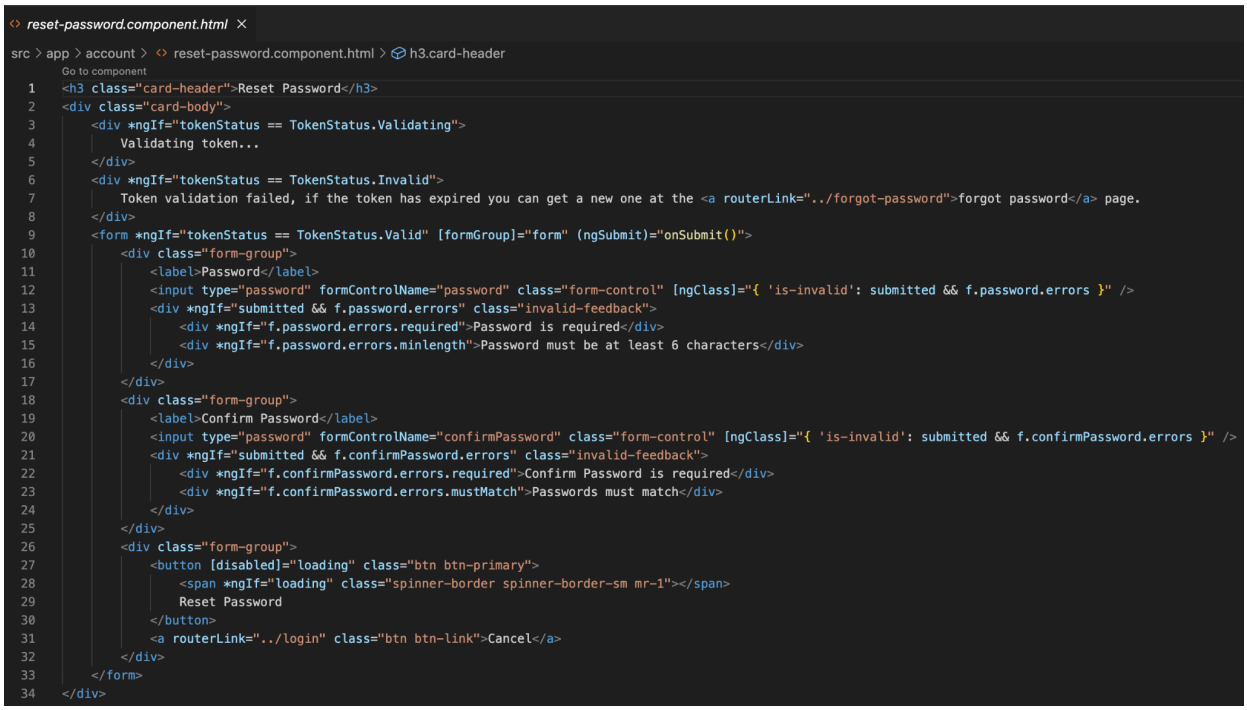
        this.loading = true;
        this.accountService.register(this.form.value)
            .pipe(first())
            .subscribe({
                next: () => {
                    this.alertService.success('Registration successful, please check your email for verification instructions', { keepAfterRouteChange: true });
                    this.router.navigate(['../login'], { relativeTo: this.route });
                },
                error: error => {
                    this.alertService.error(error);
                    this.loading = false;
                }
            });
    }
}
```

Reset Password Component Template

Path: /src/app/account/reset-password.component.html

The reset password component template renders one of the following three views based on the status of the token being validated by the reset password component:

- Token Validating: a message stating that the token is validating.
- Token Invalid: a message that the validation failed and a link to the forgot password page to get a new token.
- Token Valid: a form to reset the password that contains fields for password and confirm password.



Reset Password Component

Path: /src/app/account/reset-password.component.ts

The reset password component displays a form for resetting an account password when it receives a valid password reset `token` in the url query string parameters. The token is validated when the component initialized by calling `this.accountService.validateResetToken(token)` from the `ngOnInit()` Angular lifecycle method.

The `tokenStatus` controls what is rendered by the reset password component template, the initial status is `Validating` before changing to either `Valid` or `Invalid`. The `TokenStatus` enum is used to set the status so we don't have to use string values.

On form submit the password is reset by calling `this.accountService.resetPassword(...)` which sends the token and new password to the backend. The backend should validate the token again before updating the password, see the `resetPassword()` function in the fake backend for an example.

On successful password reset the user is redirected to the login page with a success message and can login to the boilerplate app with the new password.

TS *reset-password.component.ts* ×

src > app > account > TS reset-password.component.ts > ...

```ts
import { Component, OnInit } from '@angular/core';
import { Router, ActivatedRoute } from '@angular/router';
import { UntypedFormBuilder, UntypedFormGroup, Validators } from '@angular/forms';
import { first } from 'rxjs/operators';

import { AccountService, AlertService } from '@app/_services';
import { MustMatch } from '@app/_helpers';

enum TokenStatus {
    Validating,
    Valid,
    Invalid
}

@Component({ templateUrl: 'reset-password.component.html' })
export class ResetPasswordComponent implements OnInit {
    TokenStatus = TokenStatus;
    tokenStatus = TokenStatus.Validating;
    token = null;
    form: UntypedFormGroup;
    loading = false;
    submitted = false;

    constructor(
        private formBuilder: UntypedFormBuilder,
        private route: ActivatedRoute,
        private router: Router,
        private accountService: AccountService,
        private alertService: AlertService
    ) { }

```

```
    ngOnInit() {
        this.form = this.formBuilder.group({
            password: ['', [Validators.required, Validators.minLength(6)]],
            confirmPassword: ['', Validators.required],
        }, {
            validator: MustMatch('password', 'confirmPassword')
        });

        const token = this.route.snapshot.queryParams['token'];

        // remove token from url to prevent http referer leakage
        this.router.navigate([], { relativeTo: this.route, replaceUrl: true });

        this.accountService.validateResetToken(token)
            .pipe(first())
            .subscribe({
                next: () => {
                    this.token = token;
                    this.tokenStatus = TokenStatus.Valid;
                },
                error: () => {
                    this.tokenStatus = TokenStatus.Invalid;
                }
            });
    }

    // convenience getter for easy access to form fields
    get f() { return this.form.controls; }
```

```
    onSubmit() {
        this.submitted = true;

        // reset alerts on submit
        this.alertService.clear();

        // stop here if form is invalid
        if (this.form.invalid) {
            return;
        }

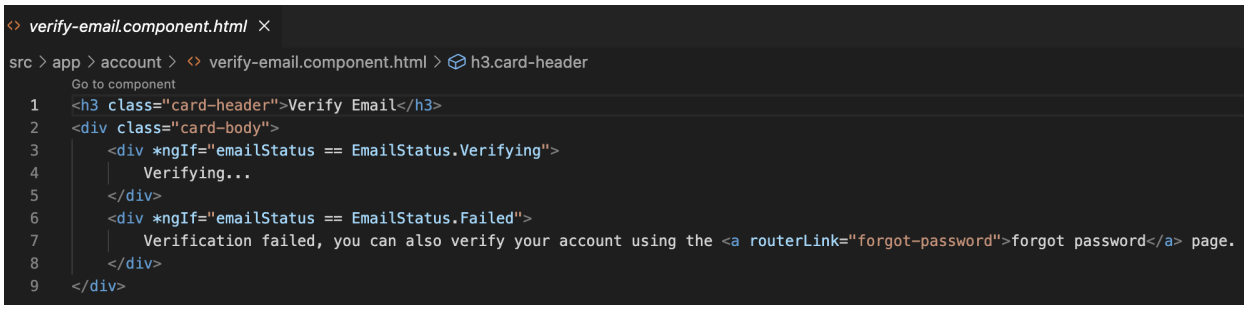
        this.loading = true;
        this.accountService.resetPassword(this.token, this.f.password.value, this.f.confirmPassword.value)
            .pipe(first())
            .subscribe({
                next: () => {
                    this.alertService.success('Password reset successful, you can now login', { keepAfterRouteChange: true });
                    this.router.navigate(['../login'], { relativeTo: this.route });
                },
                error: error => {
                    this.alertService.error(error);
                    this.loading = false;
                }
            });
    }
}
```

Verify Email Component Template

Path: /src/app/account/verify-email.component.html

The verify email component template renders one of the following two views based on the status of the email token being verified by the verify email component:

- Verifying: a message stating that the email is verifying.
- Failed: a message that email verification failed and a link to the forgot password page as an alternative way to verify the email.



Verify Email Component

Path: /src/app/account/verify-email.component.ts

The verify email component is used to verify new accounts before they can login to the boilerplate app. When a new account is registered an email is sent to the user containing a link back to this component with a verification `token` in the querystring parameters. The token from the email link is verified when the component initialized by calling

`this.accountService.verifyEmail(token)` from the `ngOnInit()` Angular lifecycle method.

On successful verification the user is redirected to the login page with a success message and can login to the account, if token verification fails an error message is displayed and a link to the forgot password page which can also be used to verify an account.

NOTE: When using the app with the fake backend the verification "email" is displayed on the screen when a new account is registered.

verify-email.component.ts

src > app > account > verify-email.component.ts > VerifyEmailComponent > ngOnInit

```ts
import { Component, OnInit } from '@angular/core';
import { Router, ActivatedRoute } from '@angular/router';
import { first } from 'rxjs/operators';

import { AccountService, AlertService } from '@app/_services';

enum EmailStatus {
    Verifying,
    Failed
}

@Component({ templateUrl: 'verify-email.component.html' })
export class VerifyEmailComponent implements OnInit {
    EmailStatus = EmailStatus;
    emailStatus = EmailStatus.Verifying;

    constructor(
        private route: ActivatedRoute,
        private router: Router,
        private accountService: AccountService,
        private alertService: AlertService
    ) { }

    ngOnInit() {
        const token = this.route.snapshot.queryParams['token'];

        // remove token from url to prevent http referer leakage
        this.router.navigate([], { relativeTo: this.route, replaceUrl: true });

        this.accountService.verifyEmail(token)
            .pipe(first())
            .subscribe({
                next: () => {
                    this.alertService.success('Verification successful, you can now login', { keepAfterRouteChange: true });
                    this.router.navigate(['../login'], { relativeTo: this.route });
                },
                error: () => {
                    this.emailStatus = EmailStatus.Failed;
                }
            });
    }
}
```

Admin » Accounts Routing Module

Path: /src/app/admin/accounts/accounts-routing.module.ts

The accounts routing module defines the routes for the accounts feature module. It includes routes for listing, adding and editing accounts. The add and edit routes are separate but both load the same component (`AddEditComponent`) which modifies its behavior based on the route.

TS accounts-routing.module.ts ×

src > app > admin > accounts > TS accounts-routing.module.ts > ...

```ts
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { ListComponent } from './list.component';
import { AddEditComponent } from './add-edit.component';

const routes: Routes = [
    { path: '', component: ListComponent },
    { path: 'add', component: AddEditComponent },
    { path: 'edit/:id', component: AddEditComponent }
];

@NgModule({
    imports: [RouterModule.forChild(routes)],
    exports: [RouterModule]
})
export class AccountsRoutingModule { }
```

Admin » Accounts Module

Path: /src/app/admin/accounts/accounts.module.ts

The accounts module defines the feature module for the accounts section along with metadata about the module. The `imports` specify which other angular modules are required by this module, and the `declarations` specify which components belong to this module. For more info about angular 10 modules see https://angular.io/docs/ts/latest/guide/ngmodule.html.

The accounts module is hooked into the Angular boilerplate app as a child of the admin section via the admin routing module with lazy loading.

TS accounts.module.ts ×

src > app > admin > accounts > TS accounts.module.ts > ...

```ts
import { NgModule } from '@angular/core';
import { ReactiveFormsModule } from '@angular/forms';
import { CommonModule } from '@angular/common';

import { AccountsRoutingModule } from './accounts-routing.module';
import { ListComponent } from './list.component';
import { AddEditComponent } from './add-edit.component';

@NgModule({
    imports: [
        CommonModule,
        ReactiveFormsModule,
        AccountsRoutingModule
    ],
    declarations: [
        ListComponent,
        AddEditComponent
    ]
})
export class AccountsModule { }
```

Admin » Accounts Add/Edit Component Template

Path: /src/app/admin/accounts/add-edit.component.html

The accounts add/edit component template contains a dynamic form that supports both creating and updating accounts. The `isAddMode` property is

used to change what is displayed based on which mode the component is in, for example the form title and password optional message.

add-edit.component.html

src > app > admin > accounts > add-edit.component.html > h1

Go to component

```
<h1 *ngIf="isAddMode">Create Account</h1>
<h1 *ngIf="!isAddMode">Edit Account</h1>
<form [formGroup]="form" (ngSubmit)="onSubmit()">
    <div class="form-row">
        <div class="form-group col">
            <label>Title</label>
            <select formControlName="title" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.title.errors }">
                <option value=""></option>
                <option value="Mr">Mr</option>
                <option value="Mrs">Mrs</option>
                <option value="Miss">Miss</option>
                <option value="Ms">Ms</option>
            </select>
            <div *ngIf="submitted && f.title.errors" class="invalid-feedback">
                <div *ngIf="f.title.errors.required">Title is required</div>
            </div>
        </div>
        <div class="form-group col-5">
            <label>First Name</label>
            <input type="text" formControlName="firstName" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.firstName.errors }" />
            <div *ngIf="submitted && f.firstName.errors" class="invalid-feedback">
                <div *ngIf="f.firstName.errors.required">First Name is required</div>
            </div>
        </div>
        <div class="form-group col-5">
            <label>Last Name</label>
            <input type="text" formControlName="lastName" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.lastName.errors }" />
            <div *ngIf="submitted && f.lastName.errors" class="invalid-feedback">
                <div *ngIf="f.lastName.errors.required">Last Name is required</div>
            </div>
        </div>
    </div>
```

```
    <div class="form-row">
        <div class="form-group col-7">
            <label>Email</label>
            <input type="text" formControlName="email" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.email.errors }" />
            <div *ngIf="submitted && f.email.errors" class="invalid-feedback">
                <div *ngIf="f.email.errors.required">Email is required</div>
                <div *ngIf="f.email.errors.email">Email must be a valid email address</div>
            </div>
        </div>
        <div class="form-group col">
            <label>Role</label>
            <select formControlName="role" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.role.errors }">
                <option value=""></option>
                <option value="User">User</option>
                <option value="Admin">Admin</option>
            </select>
            <div *ngIf="submitted && f.role.errors" class="invalid-feedback">
                <div *ngIf="f.role.errors.required">Role is required</div>
            </div>
        </div>
    </div>
    <div *ngIf="!isAddMode">
        <h3 class="pt-3">Change Password</h3>
        <p>Leave blank to keep the same password</p>
    </div>
```

```
    <div class="form-row">
        <div class="form-group col">
            <label>Password</label>
            <input type="password" formControlName="password" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.password.errors }" />
            <div *ngIf="submitted && f.password.errors" class="invalid-feedback">
                <div *ngIf="f.password.errors.required">Password is required</div>
                <div *ngIf="f.password.errors.minlength">Password must be at least 6 characters</div>
            </div>
        </div>
        <div class="form-group col">
            <label>Confirm Password</label>
            <input type="password" formControlName="confirmPassword" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.confirmPassword.errors }" />
            <div *ngIf="submitted && f.confirmPassword.errors" class="invalid-feedback">
                <div *ngIf="f.confirmPassword.errors.required">Confirm Password is required</div>
                <div *ngIf="f.confirmPassword.errors.mustMatch">Passwords must match</div>
            </div>
        </div>
    </div>
    <div class="form-group">
        <button [disabled]="loading" class="btn btn-primary">
            <span *ngIf="loading" class="spinner-border spinner-border-sm mr-1"></span>
            Save
        </button>
        <a routerLink="/admin/accounts" class="btn btn-link">Cancel</a>
    </div>
</form>
```

Admin » Accounts Add/Edit Component

Path: /src/app/admin/accounts/add-edit.component.ts

The accounts add/edit component is used for both creating and updating accounts, the component is in "add mode" when there is no account id route parameter, otherwise it is in "edit mode". The property `isAddMode` is used to change the component behavior based on which mode it is in, for example in "add mode" the password field is required, and in "edit mode" (`!this.isAddMode`) the account service is called when the component initializes to get the account details (`this.accountService.getById(this.id)`) to pre-populate the field values.

On submit a account is either created or updated by calling the account service, and on success the user is redirected back to the accounts list page with a success message.

TS add-edit.component.ts

src > app > admin > accounts > TS add-edit.component.ts > ...

```ts
import { Component, OnInit } from '@angular/core';
import { Router, ActivatedRoute } from '@angular/router';
import { UntypedFormBuilder, UntypedFormGroup, Validators } from '@angular/forms';
import { first } from 'rxjs/operators';

import { AccountService, AlertService } from '@app/_services';
import { MustMatch } from '@app/_helpers';

@Component({ templateUrl: 'add-edit.component.html' })
export class AddEditComponent implements OnInit {
    form: UntypedFormGroup;
    id: string;
    isAddMode: boolean;
    loading = false;
    submitted = false;

    constructor(
        private formBuilder: UntypedFormBuilder,
        private route: ActivatedRoute,
        private router: Router,
        private accountService: AccountService,
        private alertService: AlertService
    ) {}

    ngOnInit() {
        this.id = this.route.snapshot.params['id'];
        this.isAddMode = !this.id;

        this.form = this.formBuilder.group({
            title: ['', Validators.required],
            firstName: ['', Validators.required],
            lastName: ['', Validators.required],
            email: ['', [Validators.required, Validators.email]],
            role: ['', Validators.required],
            password: ['', [Validators.minLength(6), this.isAddMode ? Validators.required : Validators.nullValidator]],
            confirmPassword: ['']
        }, {
            validator: MustMatch('password', 'confirmPassword')
        });

        if (!this.isAddMode) {
            this.accountService.getById(this.id)
                .pipe(first())
                .subscribe(x => this.form.patchValue(x));
        }
    }

    // convenience getter for easy access to form fields
    get f() { return this.form.controls; }

```

```
    onSubmit() {
        this.submitted = true;

        // reset alerts on submit
        this.alertService.clear();

        // stop here if form is invalid
        if (this.form.invalid) {
            return;
        }

        this.loading = true;
        if (this.isAddMode) {
            this.createAccount();
        } else {
            this.updateAccount();
        }
    }

    private createAccount() {
        this.accountService.create(this.form.value)
            .pipe(first())
            .subscribe({
                next: () => {
                    this.alertService.success('Account created successfully', { keepAfterRouteChange: true });
                    this.router.navigate(['../'], { relativeTo: this.route });
                },
                error: error => {
                    this.alertService.error(error);
                    this.loading = false;
                }
            });
    }

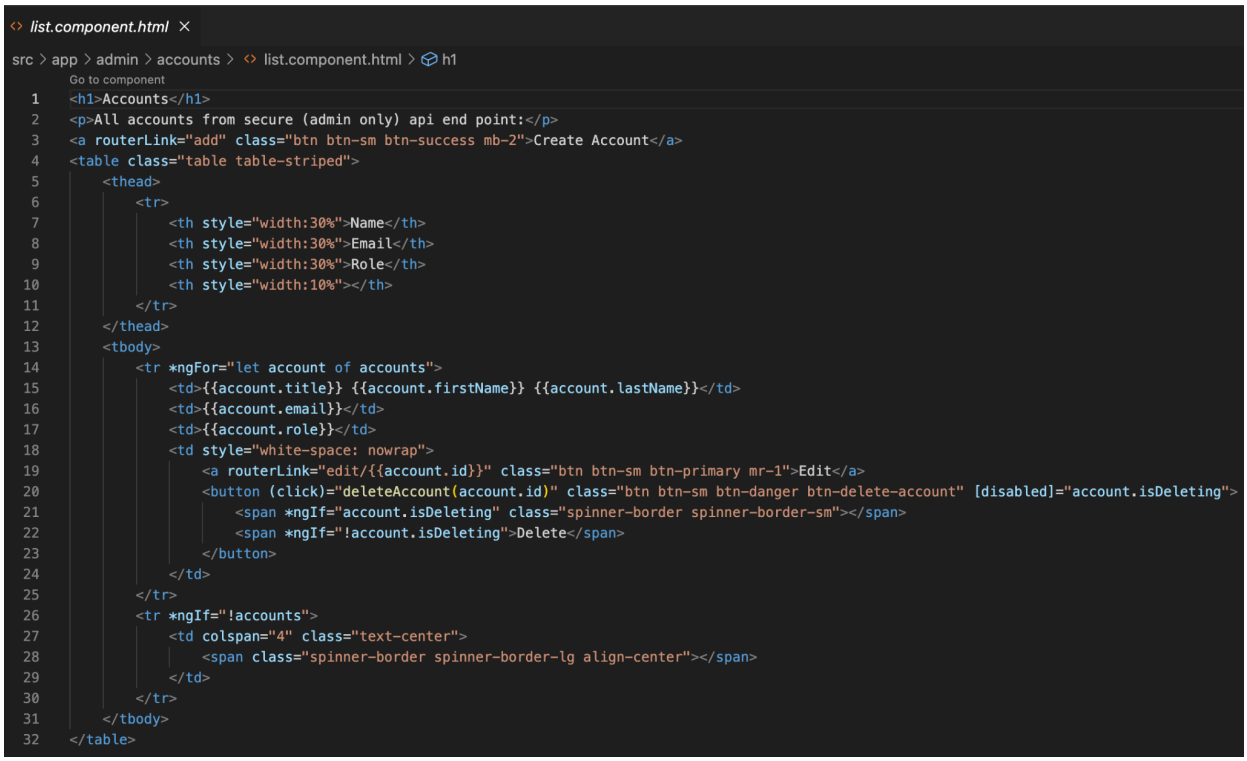
```

```
     private updateAccount() {
         this.accountService.update(this.id, this.form.value)
             .pipe(first())
             .subscribe({
                 next: () => {
                     this.alertService.success('Update successful', { keepAfterRouteChange: true });
                     this.router.navigate(['../../'], { relativeTo: this.route });
                 },
                 error: error => {
                     this.alertService.error(error);
                     this.loading = false;
                 }
             });
     }
 }
```

Admin » Accounts List Component Template

Path: /src/app/admin/accounts/list.component.html

The accounts list component template displays a list of all accounts and contains buttons for creating, editing and deleting accounts.



Admin » Accounts List Component

Path: /src/app/admin/accounts/list.component.ts

The accounts list component gets all accounts from the account service in the `ngOnInit()` method and makes them available to the accounts list template via the `accounts` property.

The `deleteAccount()` method sets the property `account.isDeleting = true` so the template displays a spinner on the delete button, then calls

`this.accountService.delete(id)` to delete the account and removes the deleted account from component `accounts` array so it is removed from the UI.

TS *list.component.ts* ×

src > app > admin > accounts > TS list.component.ts > ...

```ts
import { Component, OnInit } from '@angular/core';
import { first } from 'rxjs/operators';

import { AccountService } from '@app/_services';
import { Account } from '@app/_models';

@Component({ templateUrl: 'list.component.html' })
export class ListComponent implements OnInit {
    accounts: any[];

    constructor(private accountService: AccountService) {}

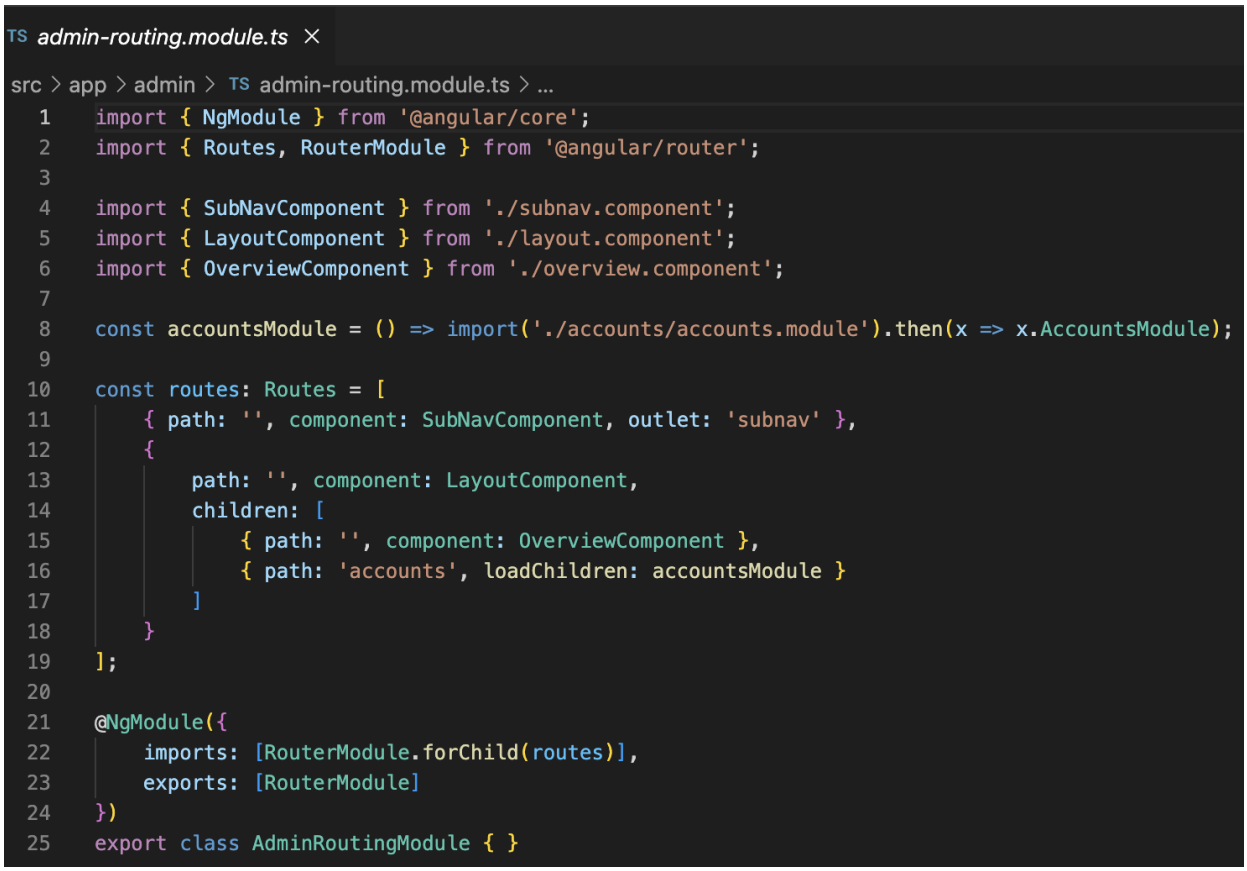
    ngOnInit() {
        this.accountService.getAll()
            .pipe(first())
            .subscribe(accounts => this.accounts = accounts);
    }

    deleteAccount(id: string) {
        const account = this.accounts.find(x => x.id === id);
        account.isDeleting = true;
        this.accountService.delete(id)
            .pipe(first())
            .subscribe(() => {
                this.accounts = this.accounts.filter(x => x.id !== id)
            });
    }
}
```

Admin Routing Module

Path: /src/app/admin/admin-routing.module.ts

The admin routing module defines the routes for the admin feature module. It includes a route for the admin subnav component that is displayed below the main nav on all admin pages, plus routes for the overview page and accounts section, and a parent route for the admin layout component which contains the common layout code for the admin section. The accounts module is lazy loaded as a child module of the admin module.



Admin Module

Path: /src/app/admin/admin.module.ts

The admin module defines the feature module for the admin section along with metadata about the module. The `imports` specify which other angular modules are required by this module, and the `declarations` specify which components belong to this module. For more info about angular 10 modules see https://angular.io/docs/ts/latest/guide/ngmodule.html.

The admin module is hooked into the main app inside the app routing module with lazy loading.

TS admin.module.ts

src > app > admin > TS admin.module.ts > ...

```ts
import { NgModule } from '@angular/core';
import { ReactiveFormsModule } from '@angular/forms';
import { CommonModule } from '@angular/common';

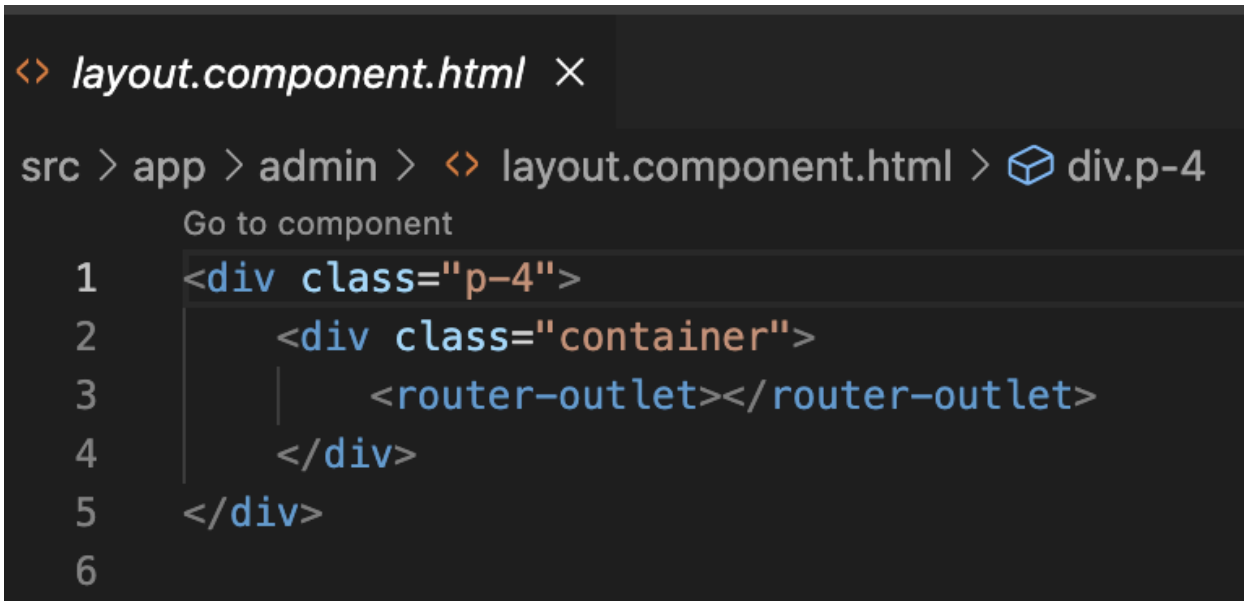
import { AdminRoutingModule } from './admin-routing.module';
import { SubNavComponent } from './subnav.component';
import { LayoutComponent } from './layout.component';
import { OverviewComponent } from './overview.component';

@NgModule({
    imports: [
        CommonModule,
        ReactiveFormsModule,
        AdminRoutingModule
    ],
    declarations: [
        SubNavComponent,
        LayoutComponent,
        OverviewComponent
    ]
})
export class AdminModule { }
```

Admin Layout Component Template

Path: /src/app/admin/layout.component.html

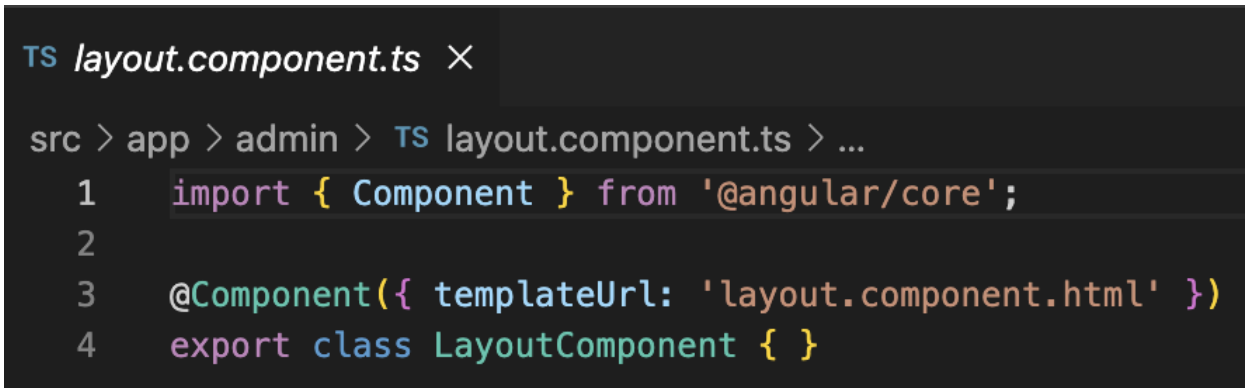
The admin layout component template is the root template of the admin feature / section of the app, it contains the outer HTML for all `/admin` pages and a `<router-outlet>` for rendering the currently routed component.



Admin Layout Component

Path: /src/app/admin/layout.component.ts

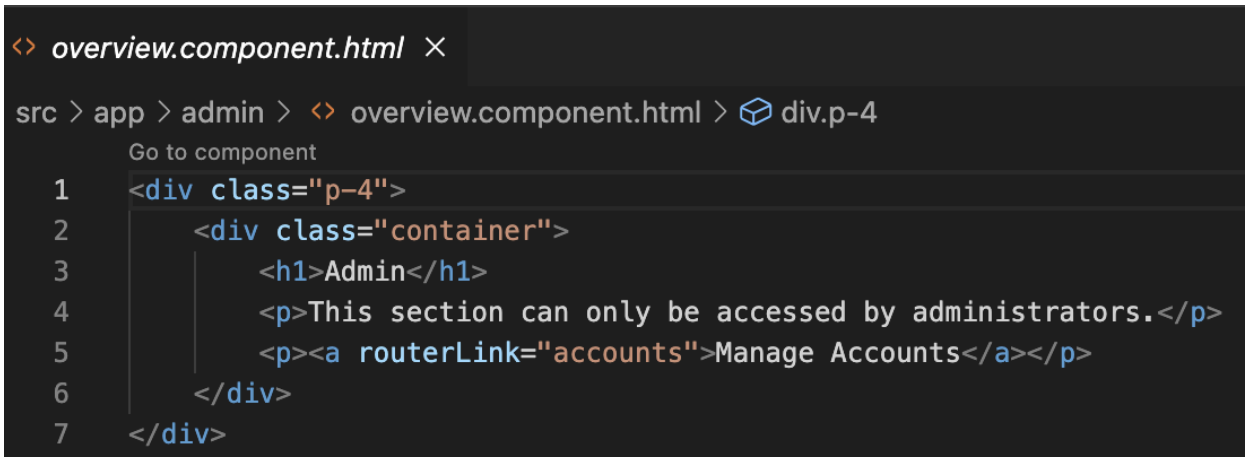
The admin layout component is the root component of the admin feature / section of the boilerplate app, it is bound to the admin layout template with the `templateUrl` property of the angular `@Component` decorator.



Admin Overview Component Template

Path: /src/app/admin/overview.component.html

The admin overview component template displays some basic HTML and a link to the account admin section.



Admin Overview Component

Path: /src/app/admin/overview.component.ts

The admin overview component is the default component of the admin section of the boilerplate app, it is bound to the admin overview template with the `templateUrl` property of the angular `@Component` decorator.

```
TS overview.component.ts ×
src > app > admin > TS overview.component.ts > ...
1   import { Component } from '@angular/core';
2
3   @Component({ templateUrl: 'overview.component.html' })
4   export class OverviewComponent { }
```

Admin Sub Nav Component Template

Path: /src/app/admin/subnav.component.html

The admin sub nav component template contains the navigation for the admin section of the boilerplate app, it is displayed directly below the main nav on all admin pages of the application.

The sub nav is rendered by the `"subnav"` router outlet in the app component template and is bound to the router outlet in the admin routing module.

```
<> subnav.component.html ×
src > app > admin > <> subnav.component.html > nav.admin-nav.navbar.navbar-expand.navbar-light
    Go to component
1   <nav class="admin-nav navbar navbar-expand navbar-light">
2       <div class="navbar-nav">
3           <a routerLink="accounts" routerLinkActive="active" class="nav-item nav-link">Accounts</a>
4       </div>
5   </nav>
```

Admin Sub Nav Component

Path: /src/app/admin/subnav.component.ts

The admin sub nav component contains the class for the admin sub nav template, it is bound to the template with the `templateUrl` property of the angular `@Component` decorator.



Home Component Template

Path: /src/app/home/home.component.html

The home component template contains html and angular 10 template syntax for displaying a simple welcome message with the first name of the logged in account.

home.component.html

src > app > home > home.component.html > div.p-4

Go to component

```html
<div class="p-4">
    <div class="container">
        <h1>Hi {{account.firstName}}!</h1>
        <p>You're logged in with Angular 10 & JWT!!</p>
    </div>
</div>
```

Home Component

Path: /src/app/home/home.component.ts

The home component defines an angular 10 component that gets the current logged in account from the account service and makes it available to the home template via the `account` property.

home.component.ts

src > app > home > home.component.ts > ...

```ts
import { Component } from '@angular/core';

import { AccountService } from '@app/_services';

@Component({ templateUrl: 'home.component.html' })
export class HomeComponent {
    account = this.accountService.accountValue;

    constructor(private accountService: AccountService) { }
}
```

TS index.ts

src > app > home > TS index.ts

```
1  export * from './home.component';
```

Profile Details Component Template

Path: /src/app/profile/details.component.html

The profile details component template displays the name and email of the authenticated account with a link to the update profile page.

details.component.html

src > app > profile > details.component.html > h1

Go to component

```
1  <h1>My Profile</h1>
2  <p>
3      <strong>Name: </strong> {{account.title}} {{account.firstName}} {{account.lastName}}<br />
4      <strong>Email: </strong> {{account.email}}
5  </p>
6  <p><a routerLink="update">Update Profile</a></p>
7
```

Profile Details Component

Path: /src/app/profile/details.component.ts

The profile details component defines an angular 10 component that gets the current logged in account from the account service and makes it available to the profile details template via the `account` property.

TS details.component.ts

src > app > profile > TS details.component.ts > ...

```ts
import { Component } from '@angular/core';

import { AccountService } from '@app/_services';

@Component({ templateUrl: 'details.component.html' })
export class DetailsComponent {
    account = this.accountService.accountValue;

    constructor(private accountService: AccountService) { }
}
```

Profile Layout Component Template

Path: /src/app/profile/layout.component.html

The profile layout component template is the root template of the profile feature / section of the boilerplate app, it contains the outer HTML for all `/profile` pages and a `<router-outlet>` for rendering the currently routed component.

layout.component.html

src > app > profile > layout.component.html > div.p-4

Go to component

```
1  <div class="p-4">
2      <div class="container">
3          <router-outlet></router-outlet>
4      </div>
5  </div>
6
```

Profile Layout Component

Path: /src/app/profile/layout.component.ts

The profile layout component is the root component of the profile feature / section of the boilerplate app, it is bound to the profile layout template with the `templateUrl` property of the angular `@Component` decorator.

layout.component.ts

src > app > profile > layout.component.ts > ...

```
1  import { Component } from '@angular/core';
2
3  @Component({ templateUrl: 'layout.component.html' })
4  export class LayoutComponent { }
```

Profile Routing Module

Path: /src/app/profile/profile-routing.module.ts

The profile routing module defines the routes for the profile feature module. It includes routes for profile details and update profile, and a parent route for the profile layout which contains the common layout code for the profile section.

TS profile-routing.module.ts ×

src > app > profile > TS profile-routing.module.ts > ...

```ts
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { LayoutComponent } from './layout.component';
import { DetailsComponent } from './details.component';
import { UpdateComponent } from './update.component';

const routes: Routes = [
    {
        path: '', component: LayoutComponent,
        children: [
            { path: '', component: DetailsComponent },
            { path: 'update', component: UpdateComponent }
        ]
    }
];

@NgModule({
    imports: [RouterModule.forChild(routes)],
    exports: [RouterModule]
})
export class ProfileRoutingModule { }
```

Profile Module

Path: /src/app/profile/profile.module.ts

The profile module defines the feature module for the profile section of the angular boilerplate app. The `imports` specify which other angular modules are required by this module and the `declarations` specify which components

belong to this module. For more info about angular 10 modules see https://angular.io/docs/ts/latest/guide/ngmodule.html.

The profile module is hooked into the main app inside the app routing module with lazy loading.

TS *profile.module.ts* ×

src > app > profile > TS profile.module.ts > ...

```ts
import { NgModule } from '@angular/core';
import { ReactiveFormsModule } from '@angular/forms';
import { CommonModule } from '@angular/common';

import { ProfileRoutingModule } from './profile-routing.module';
import { LayoutComponent } from './layout.component';
import { DetailsComponent } from './details.component';
import { UpdateComponent } from './update.component';

@NgModule({
    imports: [
        CommonModule,
        ReactiveFormsModule,
        ProfileRoutingModule
    ],
    declarations: [
        LayoutComponent,
        DetailsComponent,
        UpdateComponent
    ]
})
export class ProfileModule { }
```

Profile Update Component Template

Path: /src/app/profile/update.component.html

The profile update component template contains a form for updating the details of the currently authenticated account, changing password, or deleting the account.

update.component.html

src > app > profile > update.component.html > h1

Go to component

```
<h1>Update Profile</h1>
<form [formGroup]="form" (ngSubmit)="onSubmit()">
    <div class="form-row">
        <div class="form-group col">
            <label>Title</label>
            <select formControlName="title" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.title.errors }">
                <option value=""></option>
                <option value="Mr">Mr</option>
                <option value="Mrs">Mrs</option>
                <option value="Miss">Miss</option>
                <option value="Ms">Ms</option>
            </select>
            <div *ngIf="submitted && f.title.errors" class="invalid-feedback">
                <div *ngIf="f.title.errors.required">Title is required</div>
            </div>
        </div>
        <div class="form-group col-5">
            <label>First Name</label>
            <input type="text" formControlName="firstName" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.firstName.errors }" />
            <div *ngIf="submitted && f.firstName.errors" class="invalid-feedback">
                <div *ngIf="f.firstName.errors.required">First Name is required</div>
            </div>
        </div>
        <div class="form-group col-5">
            <label>Last Name</label>
            <input type="text" formControlName="lastName" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.lastName.errors }" />
            <div *ngIf="submitted && f.lastName.errors" class="invalid-feedback">
                <div *ngIf="f.lastName.errors.required">Last Name is required</div>
            </div>
        </div>
    </div>
    <div class="form-group">
        <label>Email</label>
        <input type="text" formControlName="email" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.email.errors }" />
        <div *ngIf="submitted && f.email.errors" class="invalid-feedback">
            <div *ngIf="f.email.errors.required">Email is required</div>
            <div *ngIf="f.email.errors.email">Email must be a valid email address</div>
        </div>
    </div>
```

```
    <h3 class="pt-3">Change Password</h3>
    <p>Leave blank to keep the same password</p>
    <div class="form-row">
        <div class="form-group col">
            <label>Password</label>
            <input type="password" formControlName="password" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.password.errors }" />
            <div *ngIf="submitted && f.password.errors" class="invalid-feedback">
                <div *ngIf="f.password.errors.required">Password is required</div>
                <div *ngIf="f.password.errors.minlength">Password must be at least 6 characters</div>
            </div>
        </div>
        <div class="form-group col">
            <label>Confirm Password</label>
            <input type="password" formControlName="confirmPassword" class="form-control" [ngClass]="{ 'is-invalid': submitted && f.confirmPassword.errors }" />
            <div *ngIf="submitted && f.confirmPassword.errors" class="invalid-feedback">
                <div *ngIf="f.confirmPassword.errors.required">Confirm Password is required</div>
                <div *ngIf="f.confirmPassword.errors.mustMatch">Passwords must match</div>
            </div>
        </div>
    </div>
    <div class="form-group">
        <button type="submit" [disabled]="loading" class="btn btn-primary mr-2">
            <span *ngIf="loading" class="spinner-border spinner-border-sm mr-1"></span>
            Update
        </button>
        <button type="button" (click)="onDelete()" [disabled]="deleting" class="btn btn-danger">
            <span *ngIf="deleting" class="spinner-border spinner-border-sm mr-1"></span>
            Delete
        </button>
        <a routerLink="../" href="" class="btn btn-link">Cancel</a>
    </div>
</form>

```

Profile Update Component

Path: /src/app/profile/update.component.ts

The profile update component enables the current user to update their profile, change their password, or delete their account. It pre-populates the form fields with the current account from the account service (`this.accountService.accountValue`), and is bound to the profile update template with the `templateUrl` property of the angular `@Component` decorator.

On successful update the user is redirected back to the profile details page with a success message. On successful delete the user is logged out of the boilerplate app with a message that the account was successfully deleted.

TS update.component.ts

src > app > profile > TS update.component.ts > ...

```ts
import { Component, OnInit } from '@angular/core';
import { Router, ActivatedRoute } from '@angular/router';
import { UntypedFormBuilder, UntypedFormGroup, Validators } from '@angular/forms';
import { first } from 'rxjs/operators';

import { AccountService, AlertService } from '@app/_services';
import { MustMatch } from '@app/_helpers';

@Component({ templateUrl: 'update.component.html' })
export class UpdateComponent implements OnInit {
    account = this.accountService.accountValue;
    form: UntypedFormGroup;
    loading = false;
    submitted = false;
    deleting = false;

    constructor(
        private formBuilder: UntypedFormBuilder,
        private route: ActivatedRoute,
        private router: Router,
        private accountService: AccountService,
        private alertService: AlertService
    ) { }

    ngOnInit() {
        this.form = this.formBuilder.group({
            title: [this.account.title, Validators.required],
            firstName: [this.account.firstName, Validators.required],
            lastName: [this.account.lastName, Validators.required],
            email: [this.account.email, [Validators.required, Validators.email]],
            password: ['', [Validators.minLength(6)]],
            confirmPassword: ['']
        }, {
            validator: MustMatch('password', 'confirmPassword')
        });
    }

    // convenience getter for easy access to form fields
    get f() { return this.form.controls; }

```

```
    onSubmit() {
        this.submitted = true;

        // reset alerts on submit
        this.alertService.clear();

        // stop here if form is invalid
        if (this.form.invalid) {
            return;
        }

        this.loading = true;
        this.accountService.update(this.account.id, this.form.value)
            .pipe(first())
            .subscribe({
                next: () => {
                    this.alertService.success('Update successful', { keepAfterRouteChange: true });
                    this.router.navigate(['../'], { relativeTo: this.route });
                },
                error: error => {
                    this.alertService.error(error);
                    this.loading = false;
                }
            });
    }

    onDelete() {
        if (confirm('Are you sure?')) {
            this.deleting = true;
            this.accountService.delete(this.account.id)
                .pipe(first())
                .subscribe(() => {
                    this.alertService.success('Account deleted successfully', { keepAfterRouteChange: true });
                });
        }
    }
}
```

App Routing Module

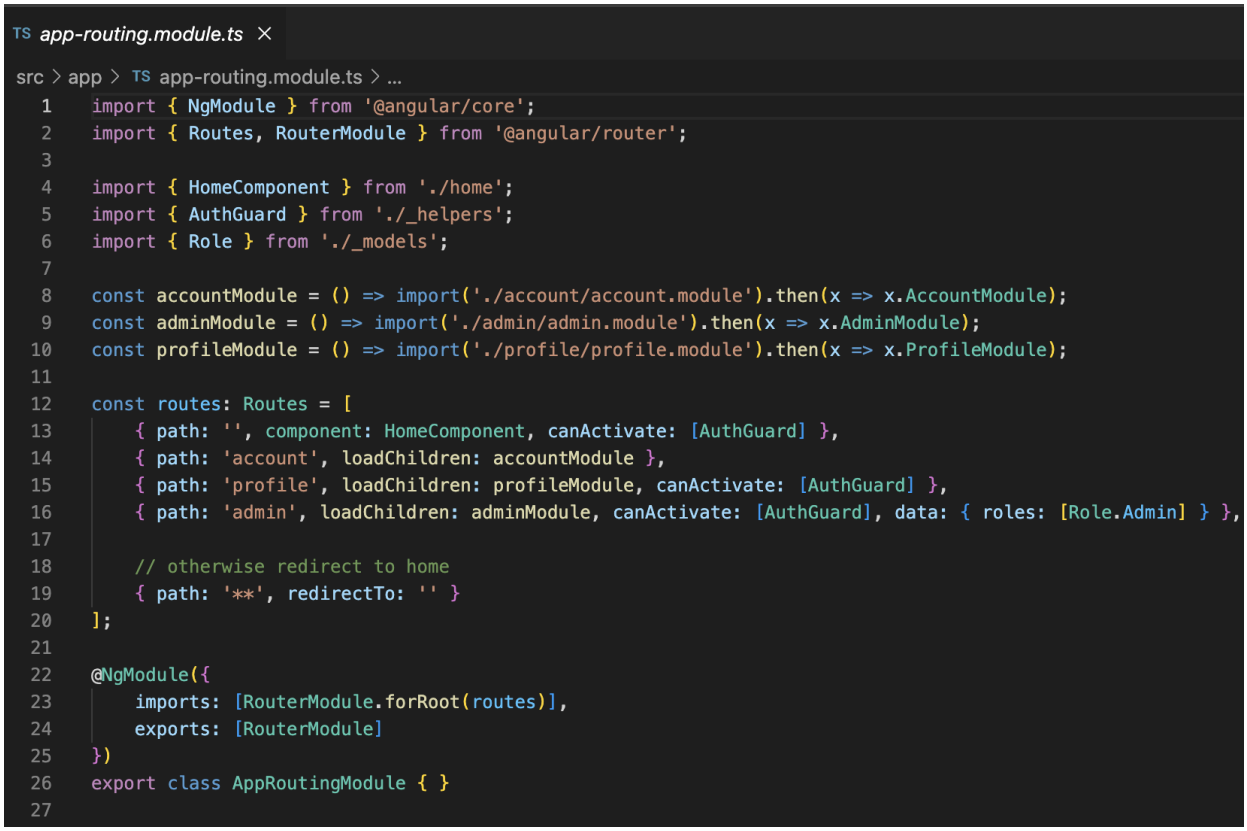
Path: /src/app/app-routing.module.ts

The app routing module defines the top level routes for the boilerplate angular application and generates a root routing module by passing the array of `routes` to the `RouterModule.forRoot()` method. The module is imported into the main app module below.

The home route maps the root path (`/`) of the app to the home component, the `/account` route maps to the account module, the `/profile` route maps to the profile module and the `/admin` route maps to the admin module, and all three feature module routes (account, profile, admin) are lazy loaded.

The home, profile and admin routes are secured by passing the auth guard to the `canActivate` property of each route, and the admin route is restricted to accounts with the `Admin` role.

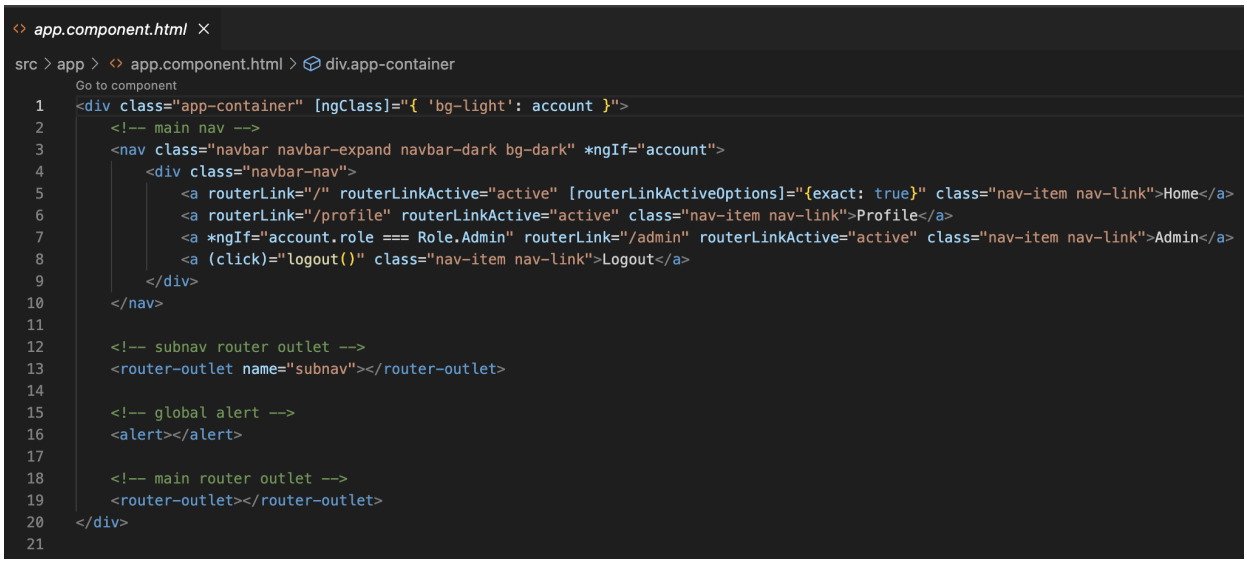
For more information on angular routing see https://angular.io/guide/router.



App Component Template

Path: /src/app/app.component.html

The app component template is the root component template of the boilerplate application, it contains the main nav bar which is only displayed for authenticated accounts, a named `router-outlet` for rendering a subnav, a global alert component, and a primary `router-outlet` displaying the contents of each view based on the current route / path.



App Component

Path: /src/app/app.component.ts

The app component is the root component of the boilerplate angular app, it defines the root tag of the app as `<app></app>` with the selector property of the `@Component()` decorator.

It subscribes to the `account` observable of the account service so it can reactively show/hide the main nav bar in the app component template when the user logs in/out of the application. I didn't worry about unsubscribing from the observable here because it's the root component of the application and will only be destroyed when the angular app is closed.

The `logout()` method is called from the logout link in the main nav bar to log the account out and redirect to the login page.

TS app.component.ts

src > app > TS app.component.ts > ...

```ts
import { Component } from '@angular/core';

import { AccountService } from './_services';
import { Account, Role } from './_models';

@Component({ selector: 'app', templateUrl: 'app.component.html' })
export class AppComponent {
    Role = Role;
    account: Account;

    constructor(private accountService: AccountService) {
        this.accountService.account.subscribe(x => this.account = x);
    }

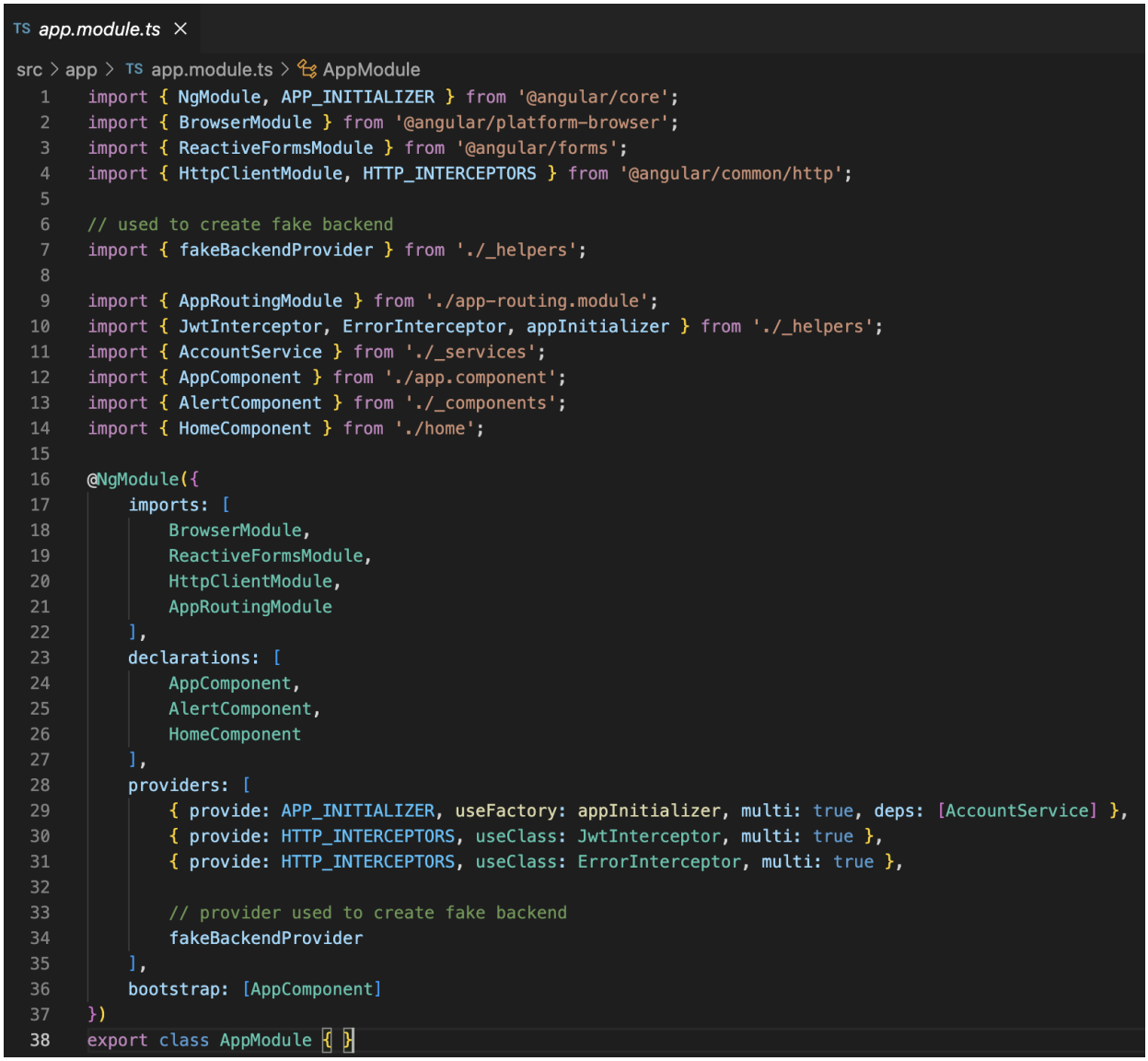
    logout() {
        this.accountService.logout();
    }
}
```

App Module

Path: /src/app/app.module.ts

The app module defines the root module of the angular boilerplate application along with metadata about the module. For more info about angular 10 modules see https://angular.io/docs/ts/latest/guide/ngmodule.html.

This is where the fake backend provider is added to the application, to switch to a real backend simply remove the `fakeBackendProvider` located below the comment `// provider used to create fake backend`.



app.module.ts

src > app > app.module.ts > AppModule

```
import { NgModule, APP_INITIALIZER } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { ReactiveFormsModule } from '@angular/forms';
import { HttpClientModule, HTTP_INTERCEPTORS } from '@angular/common/http';

// used to create fake backend
import { fakeBackendProvider } from './_helpers';

import { AppRoutingModule } from './app-routing.module';
import { JwtInterceptor, ErrorInterceptor, appInitializer } from './_helpers';
import { AccountService } from './_services';
import { AppComponent } from './app.component';
import { AlertComponent } from './_components';
import { HomeComponent } from './home';

@NgModule({
    imports: [
        BrowserModule,
        ReactiveFormsModule,
        HttpClientModule,
        AppRoutingModule
    ],
    declarations: [
        AppComponent,
        AlertComponent,
        HomeComponent
    ],
    providers: [
        { provide: APP_INITIALIZER, useFactory: appInitializer, multi: true, deps: [AccountService] },
        { provide: HTTP_INTERCEPTORS, useClass: JwtInterceptor, multi: true },
        { provide: HTTP_INTERCEPTORS, useClass: ErrorInterceptor, multi: true },

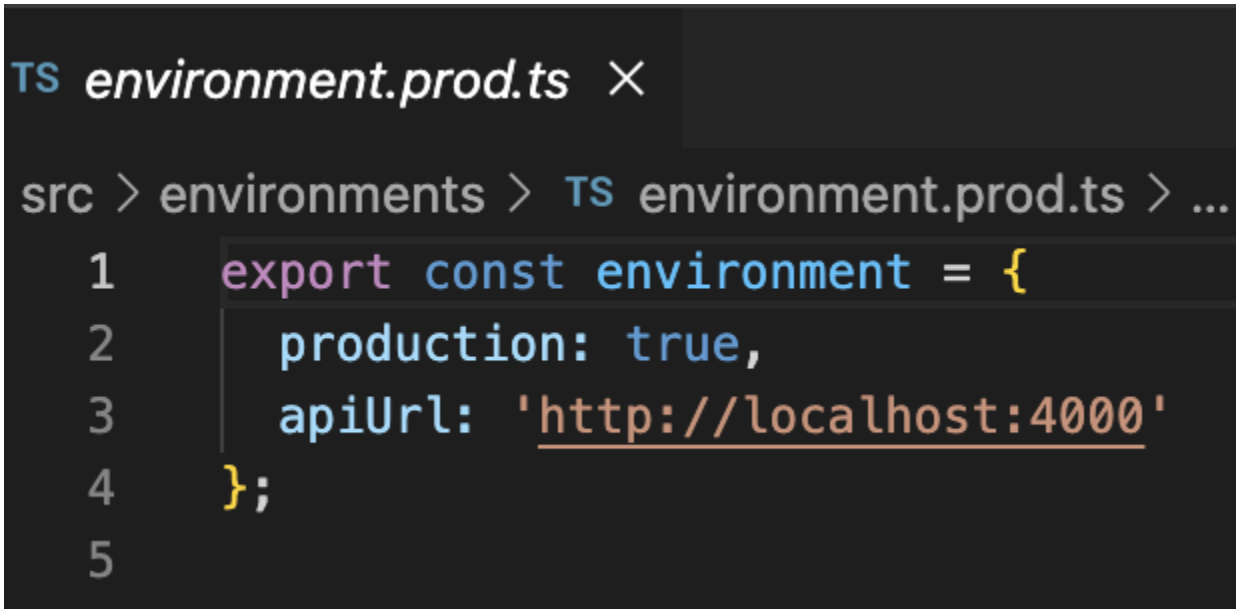
        // provider used to create fake backend
        fakeBackendProvider
    ],
    bootstrap: [AppComponent]
})
export class AppModule { }
```

Production Environment Config

Path: /src/environments/environment.prod.ts

The production environment config contains variables required to run the application in production. This enables you to build the application with a different configuration for each different environment (e.g. production & development) without updating the app code.

When you build the application for production with the command `ng build --prod`, the output `environment.ts` is replaced with `environment.prod.ts`.

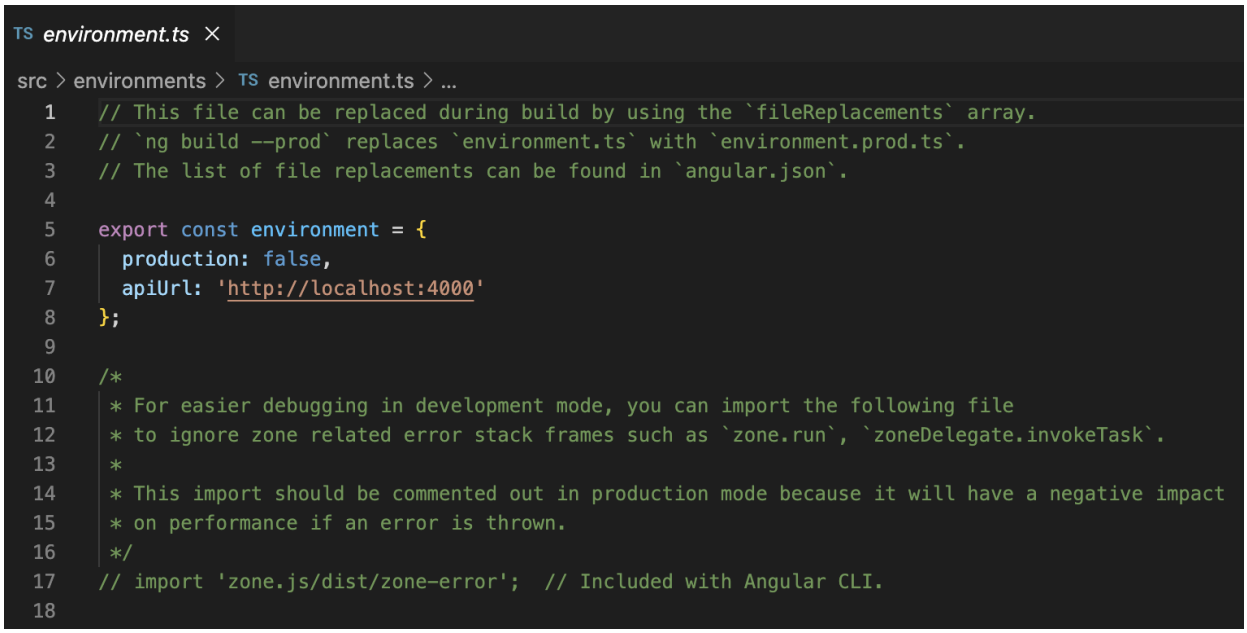


Development Environment Config

Path: /src/environments/environment.ts

The development environment config contains variables required to run the boilerplate app in development.

Environment config is accessed by importing the environment object into any Angular service or component with the line `import { environment } from '@environments/environment'` and accessing properties on the `environment` object, see the account service for an example.

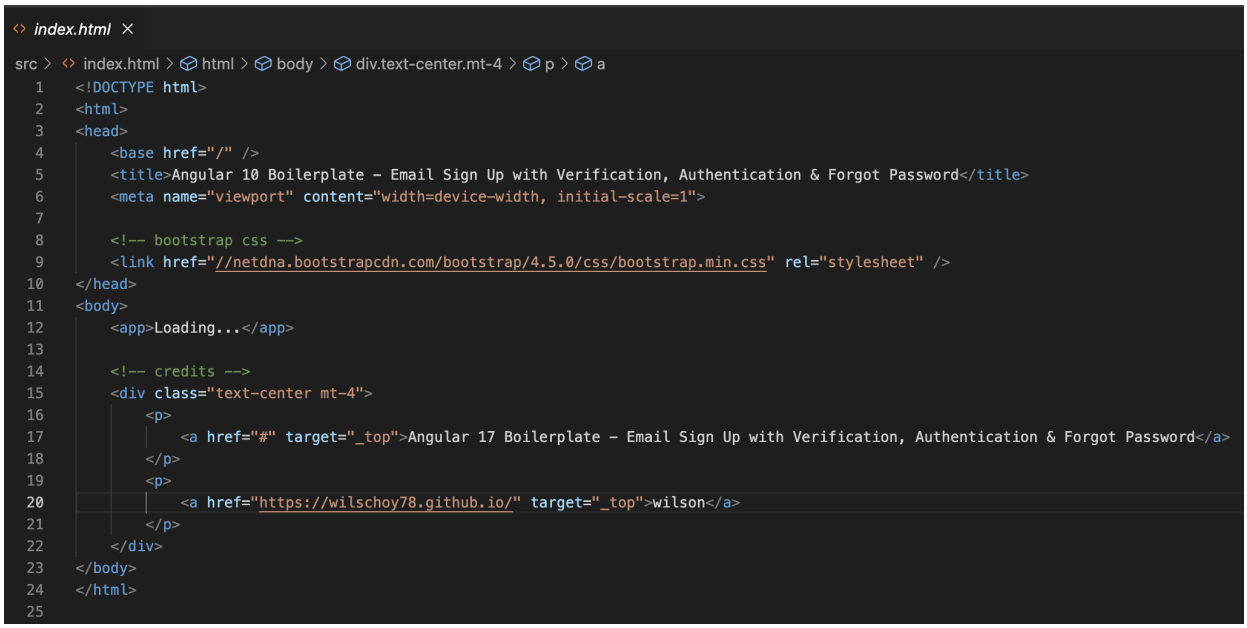


Main Index Html File

Path: /src/index.html

The main index.html file is the initial page loaded by the browser that kicks everything off. The Angular CLI (with Webpack under the hood) bundles all of

the compiled javascript files together and injects them into the body of the index.html page so the scripts can be loaded and executed by the browser.



Main (Bootstrap) File

Path: /src/main.ts

The main file is the entry point used by angular to launch and bootstrap the application.

TS main.ts

src > TS main.ts > ...

```ts
import { enableProdMode } from '@angular/core';
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';

import { AppModule } from './app/app.module';
import { environment } from './environments/environment';

if (environment.production) {
    enableProdMode();
}

platformBrowserDynamic().bootstrapModule(AppModule)
    .catch(err => console.error(err));
```

Polyfills

Path: /src/polyfills.ts

Some features used by Angular 10 are not yet supported natively by all major browsers, polyfills are used to add support for features where necessary so your Angular 10 boilerplate application works across all major browsers.

This file is generated by the Angular CLI when creating a new project with the `ng new` command, I've excluded the comments in the file for brevity.

TS polyfills.ts

src > TS polyfills.ts

```ts
*/
// import 'zone.js/dist/zone';  // Included with Angular CLI.
import 'zone.js';
```

Global LESS/CSS Styles

Path: /src/styles.less

The global styles file contains LESS/CSS styles that are applied globally throughout the angular 10 boilerplate application.

{} styles.less

src > {} styles.less > ...

```less
// global styles
a { cursor: pointer; }

.app-container {
    min-height: 320px;
}

.admin-nav {
    padding-top: 0;
    padding-bottom: 0;
    background-color: #e8e9ea;
    border-bottom: 1px solid #ccc;
}

.btn-delete-account {
    width: 40px;
    text-align: center;
    box-sizing: content-box;
}
```

Package.json

Path: /package.json

The package.json file contains project configuration information including package dependencies that get installed when you run `npm install` and scripts that are executed when you run `npm start` or `npm run build` etc. Full documentation is available at https://docs.npmjs.com/files/package.json.


{} package.json
{} package.json > ...
Debug
{
"name": "angular-signup-verification-boilerplate",
"version": "0.0.0",
"scripts": {
"ng": "ng",
"start": "ng serve --open",
"build": "ng build",
"test": "ng test",
"lint": "ng lint",
"e2e": "ng e2e"
},
"private": true,
"dependencies": {
"@angular/animations": "^17.3.2",
"@angular/common": "^17.3.2",
"@angular/compiler": "^17.3.2",
"@angular/core": "^17.3.2",
"@angular/forms": "^17.3.2",
"@angular/platform-browser": "^17.3.2",
"@angular/platform-browser-dynamic": "^17.3.2",
"@angular/router": "^17.3.2",
"rxjs": "^6.5.5",
"tslib": "^2.0.0",
"zone.js": "^0.14.4"
},

```
  "devDependencies": {
    "@angular-devkit/build-angular": "^17.3.2",
    "@angular/cli": "^17.3.2",
    "@angular/compiler-cli": "^17.3.2",
    "@types/jasmine": "^3.5.0",
    "@types/jasminewd2": "^2.0.3",
    "@types/node": "^12.11.1",
    "codelyzer": "^0.0.28",
    "jasmine-core": "^3.5.0",
    "jasmine-spec-reporter": "^5.0.0",
    "karma": "^6.4.3",
    "karma-chrome-launcher": "^3.1.0",
    "karma-coverage-istanbul-reporter": "^3.0.2",
    "karma-jasmine": "^3.3.0",
    "karma-jasmine-html-reporter": "^1.5.0",
    "protractor": "^7.0.0",
    "ts-node": "^8.3.0",
    "tslint": "^6.1.0",
    "typescript": "^5.4.3"
  }
}
```

TypeScript tsconfig.base.json

Path: /tsconfig.base.json

The tsconfig.base.json file contains the base TypeScript compiler configuration for all projects in the Angular workspace, it configures how the TypeScript code will be compiled / transpiled into JavaScript that is

understood by the browser. For more info see https://angular.io/config/tsconfig.

Most of the file is unchanged from when it was generated by the Angular CLI, only the `paths` property has been added to map the `@app` and `@environments` aliases to the `/src/app` and `/src/environments` directories. This allows imports to be relative to the app and environments folders by prefixing import paths with aliases instead of having to use long relative paths (e.g. `import MyComponent from '@app/MyComponent'` instead of `import MyComponent from '../../../MyComponent'`).


{} tsconfig.base.json
{} tsconfig.base.json > ...
/* To learn more about this file see: https://angular.io/config/tsconfig. */
{
"compileOnSave": false,
"compilerOptions": {
"baseUrl": "./",
"outDir": "./dist/out-tsc",
"sourceMap": true,
"declaration": false,
"downlevelIteration": true,
"experimentalDecorators": true,
"moduleResolution": "node",
"importHelpers": true,
"target": "es2015",
"module": "es2020",
"lib": [
"es2018",
"dom"
],
"paths": {
"@app/*": ["src/app/*"],
"@environments/*": ["src/environments/*"]
}
}
}

Other files

.gitignore

.gitignore

```
# See http://help.github.com/ignore-files/ for more about ignoring files.

# compiled output
/dist
/tmp
/out-tsc
# Only exists if Bazel was run
/bazel-out

# dependencies
/node_modules

# profiling files
chrome-profiler-events*.json
speed-measure-plugin*.json

# IDEs and editors
/.idea
.project
.classpath
.c9/
*.launch
.settings/
*.sublime-workspace

# IDE - VSCode
.vscode/*
!.vscode/settings.json
!.vscode/tasks.json
!.vscode/launch.json
!.vscode/extensions.json
.history/*

# misc
/.sass-cache
/connect.lock
/coverage
/libpeerconnection.log
npm-debug.log
yarn-error.log
testem.log
/typings

# System Files
.DS_Store
Thumbs.db
```

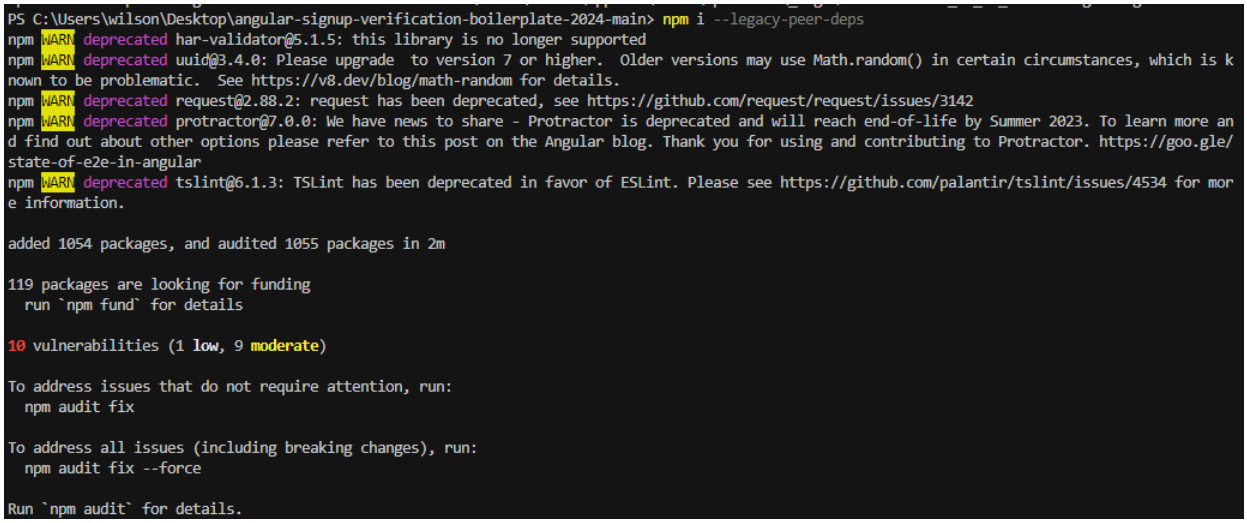
Troubleshooting:

1. During npm install you may encounter an error like this:

```
PS C:\Users\wilson\Desktop\angular-10-boilerplate> npm i
npm WARN ERESOLVE overriding peer dependency
npm WARN While resolving: angular-signup-verification-boilerplate@0.0.0
npm WARN Found: @angular/compiler-cli@10.0.14
npm WARN node_modules/@angular/compiler-cli
npm WARN   dev @angular/compiler-cli@"^11.2.14" from the root project
npm WARN
npm WARN Could not resolve dependency:
npm WARN peer @angular/compiler-cli@"^17.0.0" from @angular-devkit/build-angular@17.3.3
npm WARN node_modules/@angular-devkit/build-angular
npm WARN   dev @angular-devkit/build-angular@"^17.3.3" from the root project
npm WARN ERESOLVE overriding peer dependency
npm notice
npm notice New patch version of npm available! 10.5.1 -> 10.5.2
npm notice Changelog: https://github.com/npm/cli/releases/tag/v10.5.2
npm notice Run npm install -g npm@10.5.2 to update!
npm notice
npm ERR! code ERESOLVE
npm ERR! ERESOLVE could not resolve
npm ERR!
npm ERR! While resolving: @angular-devkit/build-angular@17.3.3
npm ERR! Found: protractor@3.3.0
npm ERR! node_modules/protractor
npm ERR!   dev protractor@"^3.3.0" from the root project
npm ERR!
npm ERR! Could not resolve dependency:
npm ERR! peerOptional protractor@"^7.0.0" from @angular-devkit/build-angular@17.3.3
npm ERR! node_modules/@angular-devkit/build-angular
npm ERR!   dev @angular-devkit/build-angular@"^17.3.3" from the root project
npm ERR!
npm ERR! Conflicting peer dependency: protractor@7.0.0
```

```
npm ERR! Conflicting peer dependency: protractor@7.0.0
npm ERR! node_modules/protractor
npm ERR!   peerOptional protractor@"^7.0.0" from @angular-devkit/build-angular@17.3.3
npm ERR!   node_modules/@angular-devkit/build-angular
npm ERR!     dev @angular-devkit/build-angular@"^17.3.3" from the root project
npm ERR!
npm ERR! Fix the upstream dependency conflict, or retry
npm ERR! this command with --force or --legacy-peer-deps
npm ERR! to accept an incorrect (and potentially broken) dependency resolution.
npm ERR!
npm ERR!
npm ERR! For a full report see:
npm ERR! C:\Users\wilson\AppData\Local\npm-cache\_logs\2024-04-20T02_39_43_690Z-eresolve-report.txt
```

2. As the npm suggested to Fix the upstream dependency conflict, or retry this command with --force or --legacy-peer-deps to accept an incorrect (and potentially broken) dependency resolution.



3. To run this without installation of angular cli or you may encounter when typing **npm start** type **npx ng serve** instead